CRYPTO QUANT LAB

SNIPER v10.10

Especificação Técnica Definitiva — Revisão 10

Sistema Quantitativo Híbrido de 5 Camadas: Market Impact √(ordem/volume), Corwin-Schultz CB, CVaR Stress ρ=1, Calibração Isotônica com Time-Decay

Fevereiro 2026 • Capital: R$ 200.000 • Horizonte: 3–5 anos

REVISÕES INCORPORADAS NESTA VERSÃO (v10.9 → v10.10)

- **1 — Market Impact pela Lei da Raiz Quadrada (NOVO — Parte 5):** Slippage linear com impact_factor estático viola modelos Kyle/Almgren-Chriss: liquidez esgota de forma quadrática. Corrigido: ΔP = σ_intraday × √(order_size / volume[t]). Slippage cresce com √(participação de mercado), não linearmente. CVaR alimentado com perda dependente de liquidez real. Aprovado.
- **2 — Corwin-Schultz como Proxy de Liquidez no Circuit Breaker (NOVO — Parte 11):** OFI de barras OHLCV 4h é ~90% ruído branco — requer dados tick-by-tick (L1) para classificação bid/ask. Circuit Breaker substituído: spread implícito de Corwin-Schultz derivado de High/Low de barras adjacentes. Pico no spread sinaliza que market makers retiraram liquidez — detectável com dados OHLCV padrão. Aprovado — correção técnica precisa.
- **3 — Stress Test ρ=1.0 como Limite Adicional ao CVaR Histórico (NOVO — Parte 10):** CVaR histórico assume que o futuro repete co-movimentos passados. Em crises sistêmicas, tail dependence → 1.0 independente do histórico. Adicionado: CVaR_stress = Σ(HRP_weights × CVaR_individual) com ρ=1.0 forçado. Se stress scenario violar limite de 18%, posições cortadas antes da ordem — preparação para cisne negro ausente do histórico. Aprovado.
- **4 — Time-Decay na Calibração Isotônica Pooled (NOVO — Partes 8 e 10):** Pooling OOS de 2020-2025 assume que score 0.85 em 2021 tem a mesma frequência real que em 2025 — microestrutura evolui, alpha decai. IsotonicRegression aceita sample_weight: decay exponencial com halflife=180 dias força calibrador a refletir eficiência de mercado atual, não média de 5 anos. Aprovado.

REVISÕES ANTERIORES MANTIDAS

- v10.1: Fractional Differentiation, HMM filtro primário, CPCV, sample weights por unicidade, DSR.
- v10.2: N efetivo com seleção de modelo (Logística vs LGBM), CFI, HMM sem hindsight bias.
- v10.3: KS Drift Monitor, protocolo de retreinamento, Binance Executor, Notificações.

- v10.4: VI no CFI, C2ST multivariado como alarme primário, ortogonalização não-linear das IAs.
- v10.5: Triple-Barrier Method, FracDiff expanding window, sizing contínuo por P_meta, C2ST com downsampling e PR-AUC.
- v10.6: Label Uniqueness dinâmica via t_touch, FracDiff weight cutoff τ=1e-5, Block Bootstrap no C2ST, pesos assimétricos W_SL=2×W_TS.
- v10.7: PCA expanding window antes do HMM, stacking ortogonal via Purged K-Fold, FracDiff em log(preço), Kelly CVaR-99%.
- v10.8: Triple-Barrier HLC, P_meta calibrado isotonicamente, PCA com RobustScaler+Winsorização, DSR com n_trials ≥ 5.000.
- v10.9: Slippage dinâmico linear, calibração isotônica pooled por VI, CVaR portfólio conjunto, Circuit Breaker OFI/Funding (4-12h).

Parâmetro	Valor
Capital inicial	R$ 200.000
Alavancagem	Zero — sem exceção
Retorno alvo	15–25% a.a. líquido
Drawdown máximo	18% (halt automático)
Horizonte mínimo	3–5 anos
Status	Backtest anti-fraude ANTES de capital real

PARTE 1 — O EDGE CAUSAL

1 Por que Outro Agente Perde Dinheiro para Você?

RESPOSTA DEFINITIVA

Você nunca é forçado a operar. Vários dos seus contrapartes são. Assimetria estrutural inerente ao design dos produtos e aos incentivos dos participantes.

Contraparte	Restrição Estrutural	Engine que Captura
Trader alavancado (10–20x)	Liquidação automática sem escolha de timing	Engine MR (Mean Reversion)
Insider com vesting schedule	Tokens liberam em datas contratuais imutáveis	Filtro UPS (Unlock Pressure Score)
Capital indexado / rebalanceador	Mandato exige comprar/vender em datas fixas	Engine TF (Trend Following)

PARTE 2 — UNIVERSO DE ATIVOS

2 Universo Anti-Survivorship Bias (Point-in-Time)

Regra	Especificação	Razão
Fonte	CoinGecko Historical API point-in-time	Evita survivorship bias
Universo base	Top 50 por market cap NAQUELE DIA HISTÓRICO	Ranking atual exclui colapsados
Exclusões fixas	BTC, ETH, stablecoins, wrapped tokens, LSTs	Regimes incompatíveis
Volume mínimo	Volume diário médio 30d > $20M USD	Slippage controlável
Idade mínima	> 18 meses de existência no sinal	Tokens novos têm manipulação severa
OBRIGATÓRIO	Incluir Luna, FTT, CEL, LUNA2 e todos colapsados	Sem isso o backtest é ilusão

2.1 Unlock Pressure Score (UPS)

```
category_weight =
{"Team/Founders":1.5,"VC/Investors":1.2,"Ecosystem":0.8,"Airdrop":0.6}
for ativo in universo:
    ups[ativo] = sum(pct_supply_unlock_30d[ativo][cat]*category_weight[cat] for
cat in category_weight)
universo_filtrado = [a for a in universo if ups[a] <=
np.percentile(list(ups.values()),75)]
```

PARTE 3 — FEATURE ENGINES (REVISADO v10.7: FRACDIFF NO ESPAÇO LOGARÍTMICO)

3 Feature Engines: FracDiff em log(preço), Weight Cutoff τ=1e-5, Expanding Window

DOIS ERROS CORRIGIDOS NESTA PARTE (v10.6 + v10.7)

v10.6: weight cutoff τ=1e-5 em vez de window=10 — preserva memória de longo prazo real (~500 obs efetivos vs 10). v10.7: FracDiff aplicada sobre log(preços), não preços brutos — coeficientes da série de Taylor aplicados em variações relativas (log-retornos) invariantes à escala, não em magnitudes de dólares que criam heterocedasticidade artificial.

3.1 Por que FracDiff deve ser aplicada em log(preço)

O erro matemático silencioso do espaço linear

BTC em $3.000 em 2019. BTC em $70.000 em 2024. Fator de escala: 23×.

Um movimento de 5% em 2019 = $150. Um movimento de 5% em 2024 = $3.500.

Os coeficientes da série de Taylor (w_1, w_2, ...) são ESCALARES aplicados sobre VALORES ABSOLUTOS em dólares.

No espaço linear: a variância de 2024 domina completamente os dados de 2019. A série diferenciada é heterocedástica por construção — viola a premissa de estacionariedade que a FracDiff devia garantir.

No espaço log: log(70000) - log(3000) = 3.15. Um movimento de 5% em qualquer época = log(1.05) ≈ 0.049. Invariante à escala.

Regra: fracdiff_series recebe SEMPRE np.log(prices), nunca prices brutos.

3.2 FracDiff: Weight Cutoff + Log-Espaço — Implementação Completa

```
from statsmodels.tsa.stattools import adfuller
import numpy as np, pandas as pd

def fracdiff_weights(d: float, tau: float = 1e-5) -> np.ndarray:
    """
    Pesos da serie de Taylor para diferenciacao fracionaria.
    Expande (1-L)^d ate |w_k| < tau.
    tau=1e-5: janela efetiva ~500 obs para d=0.4 (vs 10 com window=10).
    Referencia: Lopez de Prado, AFML 2018, Cap. 5.
    """
    weights = [1.0]
    k = 1
    while True:
```

```
        w = -weights[-1] * (d - k + 1) / k
        if abs(w) < tau:
            break
        weights.append(w)
        k += 1
    return np.array(weights[::-1])  # mais antigo primeiro

def fracdiff_log(prices: np.ndarray, d: float, tau: float = 1e-5) -> np.ndarray:
    """
    Diferenciacao fracionaria NO ESPACO LOGARITMICO.

    CORRETO (v10.7):   fracdiff_log(np.log(prices), d, tau)
    INCORRETO (v10.6): fracdiff_log(prices, d, tau)  -- magnitudes absolutas em
dolares

    Recebe: log_prices = np.log(prices) -- variacao percentual invariante a escala
    Retorna: serie com NaN nos primeiros (len(weights)-1) pontos.
    """
    log_p = np.log(prices)  # garantia: sempre converter para log AQUI
    w     = fracdiff_weights(d, tau=tau)
    n_w   = len(w)
    n     = len(log_p)
    result = np.full(n, np.nan)
    for t in range(n_w - 1, n):
        result[t] = np.dot(w, log_p[t - n_w + 1 : t + 1])
    return result

def find_optimal_d_expanding(price_series: pd.Series,
                             min_train_obs: int = 252,
                             pval_threshold: float = 0.05,
                             tau: float = 1e-5) -> pd.Series:
    """
    d* otimo com expanding window estrita (v10.5) +
    weight cutoff tau (v10.6) +
    espaco logaritmico (v10.7).

    d*_t calculado apenas com precos de t0 ate t-1.
    A busca por d* e feita sobre a serie log-diferenciada:
    adfuller testa estacionariedade de fracdiff_log(prices_t, d).
    """
    n         = len(price_series)
    d_optimal = pd.Series(index=price_series.index, dtype=float)

    for t in range(min_train_obs, n):
        prices_t = price_series.iloc[:t].values
        d_found  = None

        for d in np.linspace(0.05, 1.0, 20):
            # FracDiff no espaco log para teste ADF
            diff   = fracdiff_log(prices_t, d=d, tau=tau)
            clean  = diff[~np.isnan(diff)]
            if len(clean) < 30:
                continue
            try:
                if adfuller(clean)[1] < pval_threshold:
                    d_found = d
                    break
```

```
            except Exception:
                continue

        d_optimal.iloc[t] = d_found if d_found is not None else 1.0

    return d_optimal.ffill().bfill()

def compute_fracdiff_features_v107(price_store: dict,
                                   min_train_obs: int = 252,
                                   tau: float = 1e-5) -> dict:
    """
    FracDiff com todas as correcoes acumuladas:
    - Expanding window estrita para d* (v10.5): sem look-ahead no hiperparametro
    - Weight cutoff tau=1e-5 (v10.6): memoria de longo prazo real (~500 obs)
    - Espaco logaritmico (v10.7): invariante a escala de preco

    Custo: ~60-90 min para 20 ativos x 1500 dias.
    Salvar em disco na Fase 1. Recalcular mensalmente (rolling update).
    z-score rolling 252d para evitar look-ahead adicional no zscore.
    """
    feature_store = {}
    for ativo, prices in price_store.items():
        print(f"FracDiff v10.7 (log + weight cutoff + expanding): {ativo}...")
        d_series  = find_optimal_d_expanding(prices, min_train_obs=min_train_obs,
tau=tau)
        fd_values = np.full(len(prices), np.nan)

        for t in range(min_train_obs, len(prices)):
            d_t      = d_series.iloc[t]
            prices_t = prices.iloc[:t].values
            # ESPACO LOGARITMICO: fracdiff recebe vetor de precos brutos
            # e converte internamente para log -- nunca passar log externamente
            # para manter reproducibilidade e evitar duplo-log
            diff_t   = fracdiff_log(prices_t, d=d_t, tau=tau)
            fd_values[t] = diff_t[-1]

        fd_series = pd.Series(fd_values, index=prices.index)
        fd_mean   = fd_series.rolling(252, min_periods=30).mean()
        fd_std    = fd_series.rolling(252, min_periods=30).std()

        feature_store[ativo] = {
            'fracdiff_zscore'   : (fd_series - fd_mean) / (fd_std + 1e-10),
            'fracdiff_accel'    : fd_series.diff(1),
            'd_optimal_series'  : d_series,
            'tau_used'          : tau,
            'log_space'         : True,   # flag para auditoria
        }
    return feature_store

# DIAGNOSTICO COMPARATIVO v10.7 vs v10.6:
# Para cada ativo, comparar fracdiff_zscore calculado em preco bruto vs log-preco.
# Esperado em assets com forte tendencia de preco (BTC, ETH):
#   - Versao linear (errada): serie com variancia crescente ao longo do tempo
#   - Versao log (correta): serie com variancia estavel atraves dos ciclos
# Se as series forem similares: d* perto de 1.0 (quase primeira diferenca)
#   -- a transformacao log e menos critica nesse caso
# Se forem muito diferentes: o erro de heterocedasticidade era real e material
```

3.3 Tabela de Features (v10.7)

Feature	Tipo	Uso	Log-Espaço?	Expanding?	Weight Cutoff?
fracdiff_zscore	Série estacionária memória real	Feature ML primária	SIM v10.7	SIM v10.5	SIM v10.6 τ=1e-5
fracdiff_accel	Taxa de mudança	Feature ML	SIM v10.7	SIM v10.5	SIM v10.6
ret_1d / ret_5d / ret_20d	Momentum	Features ML + engines	N/A	N/A	N/A
vol_ratio / funding_rate_ma7d / basis_3m	Derivativos + Volume	Features ML	N/A	N/A	N/A
hmm_prob_bull	Regime ML	Filtro + feature	N/A	N/A	N/A
dvol_zscore / stablecoin_chg30	Macro	Features ML + regime	N/A	N/A	N/A
unlock_pressure_rank	On-chain	Feature ML	N/A	N/A	N/A
sigma_ewma	Volatilidade EWMA	Base Triple-Barrier	N/A	N/A	N/A

PARTE 4 — FILTRO DE REGIME: PCA ROBUSTO + WINSORIZAÇÃO (REVISADO v10.10)

4 Filtro de Regime: Winsorização → RobustScaler → PCA Walk-Forward → HMM

PROBLEMA CORRIGIDO v10.8: PCA Frágil a Outliers (Norma L2)

StandardScaler divide pelo desvio padrão GLOBAL — que em crypto inclui eventos de 10-20σ (Março 2020: retorno diário −50%, FTX Novembro 2022: volatilidade +800%). PCA minimiza erro L2 (eleva ao quadrado): um único crash rotaciona violentamente PC1 para apontar na direção do evento extremo, não na estrutura de volatilidade contínua. O HMM subsequente recebe componentes principais contaminados por cisnes negros isolados. Correção: Winsorização 1%-99% antes do scaler + RobustScaler (mediana + IQR) em vez de StandardScaler.

4.1 Pipeline Robusto: Winsorização → RobustScaler → PCA

Por que StandardScaler falha em crypto

FTX crash (Nov 2022): funding_rate_ma7d atingiu +15σ em um único dia. basis_3m colapsa −12σ.

StandardScaler(X): (X - mean) / std. O std inclui esses outliers — todos os outros pontos ficam comprimidos em um intervalo minúsculo.

PCA em X_scaled: os autovetores são calculados minimizando $\Sigma(\text{erro})^2$. Erro ao quadrado amplifica o outlier de 15σ por fator 225×.

Resultado: PC1 aponta na 'direção do colapso FTX'. HMM aprende estados que são na verdade 'FTX vs. não-FTX', não 'bull vs. bear'.

RobustScaler: (X - median) / IQR. A mediana e o IQR são imunes a valores extremos. Outliers de 15σ não distorcem a transformação.

Winsorização 1%-99% ANTES do scaler: cap explícito nos extremos. Garante que mesmo um outlier de 50σ seja tratado como o percentil 99.

4.2 Conta de Parâmetros HMM (inalterada v10.7)

Configuração	Parâmetros HMM	N amostras	Ratio N/params	Estabilidade
9 features, 2 estados, cov full (v10.6)	~112	~1500	≈13	INSTÁVEL
2 PCs (robusto), 2 estados, cov full (v10.8)	~12	~1500	≈125	ESTÁVEL

Configuração	Parâmetros HMM	N amostras	Ratio N/params	Estabilidade
3 PCs (robusto), 2 estados, cov full (v10.8 alt.)	~18	~1500	≈83	ESTÁVEL

4.3 Implementação: Pipeline Robusto Expanding Window

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import RobustScaler
from hmmlearn.hmm import GaussianHMM
import numpy as np, pandas as pd

HMM_FEATURES = [
    'ret_1d','ret_5d','realized_vol_30d','vol_ratio',
    'funding_rate_ma7d','basis_3m','stablecoin_chg30',
    'btc_ma200_flag','dvol_zscore'
]
N_PCA_COMPONENTS    = 2
MIN_VARIANCE_EXPLAINED = 0.80
WINSOR_LOW  = 0.01   # percentil 1%
WINSOR_HIGH = 0.99   # percentil 99%

def winsorize_array(X: np.ndarray,
                    low_pct: float = WINSOR_LOW,
                    high_pct: float = WINSOR_HIGH) -> np.ndarray:
    """
    Winsorização feature-wise: cap em percentis low_pct e high_pct.
    Calculados SOMENTE nos dados de treino. Aplicados em treino e teste.

    Garante que eventos de 50 sigma sejam tratados como percentil 99.
    Nao elimina o evento -- apenas limita seu impacto no PCA.
    """
    X_clip = X.copy()
    for col in range(X.shape[1]):
        lo = np.percentile(X[:, col], low_pct * 100)
        hi = np.percentile(X[:, col], high_pct * 100)
        X_clip[:, col] = np.clip(X[:, col], lo, hi)
    return X_clip

def fit_robust_pca(train_features: np.ndarray,
                   n_components: int = N_PCA_COMPONENTS) -> tuple:
    """
    Pipeline robusto: Winsorização -> RobustScaler -> PCA.
    Tudo fittado SOMENTE nos dados de treino (expanding window).

    v10.7: StandardScaler -> PCA (fragil a outliers)
    v10.8: Winsorização + RobustScaler -> PCA (robusto a cisnes negros)

    RobustScaler: centra pela mediana, escala pelo IQR.
    Imune a outliers de 10-20 sigma (FTX, Março 2020).

    Retorna (winsor_bounds, scaler, pca, var_explained).
    """
    # 1. Calcular bounds de winsorização no treino
```

```
    winsor_bounds = []
    for col in range(train_features.shape[1]):
        lo = np.percentile(train_features[:, col], WINSOR_LOW * 100)
        hi = np.percentile(train_features[:, col], WINSOR_HIGH * 100)
        winsor_bounds.append((lo, hi))

    # 2. Aplicar winsorização com bounds do treino
    X_win = train_features.copy()
    for col, (lo, hi) in enumerate(winsor_bounds):
        X_win[:, col] = np.clip(train_features[:, col], lo, hi)

    # 3. RobustScaler (mediana + IQR) -- robusto a outliers
    scaler = RobustScaler()
    X_scaled = scaler.fit_transform(X_win)

    # 4. PCA
    pca = PCA(n_components=n_components, random_state=42)
    pca.fit(X_scaled)
    var_exp = pca.explained_variance_ratio_.cumsum()[-1]

    return winsor_bounds, scaler, pca, var_exp

def transform_robust_pca(features: np.ndarray,
                         winsor_bounds: list, scaler, pca) -> np.ndarray:
    """
    Transforma features NOVAS com pipeline fittado no treino.
    Aplica os mesmos bounds de winsorização do treino (nunca re-calcular).
    """
    X_clip = features.copy()
    for col, (lo, hi) in enumerate(winsor_bounds):
        X_clip[:, col] = np.clip(features[:, col], lo, hi)
    X_scaled = scaler.transform(X_clip)
    return pca.transform(X_scaled)

def fit_hmm_with_robust_pca(train_features: np.ndarray,
                            train_returns: np.ndarray,
                            n_components: int = N_PCA_COMPONENTS,
                            n_hmm_states: int = 2) -> tuple:
    """
    Pipeline completo v10.8: Winsorização -> RobustScaler -> PCA -> HMM.
    Expanding window: tudo fittado nos dados de treino ate t-1.
    """
    winsor_bounds, scaler, pca, var_exp = fit_robust_pca(train_features,
n_components)

    if var_exp < MIN_VARIANCE_EXPLAINED:
        print(f"AVISO: PCA {n_components} PCs explica {var_exp:.1%}. Considerar +1
componente.")

    X_pca = pca.transform(scaler.transform(
        np.column_stack([np.clip(train_features[:, c], *winsor_bounds[c])
                         for c in range(train_features.shape[1])])
    ))

    hmm = GaussianHMM(n_components=n_hmm_states, covariance_type="full",
                      n_iter=300, random_state=42)
    hmm.fit(X_pca)
```

```
    states   = hmm.predict(X_pca)
    bull_st  = int(np.argmax([train_returns[states==s].mean() for s in
range(n_hmm_states)]))

    from sklearn.metrics import f1_score
    probs = hmm.predict_proba(X_pca)[:, bull_st]
    best_thr, best_f1 = 0.5, 0.0
    for thr in np.linspace(0.20, 0.80, 25):
        preds = (probs > thr).astype(int)
        if preds.sum() > 0:
            score = f1_score(train_returns > 0, preds, zero_division=0)
            if score > best_f1:
                best_f1, best_thr = score, thr

    return winsor_bounds, scaler, hmm, bull_st, best_thr, var_exp, pca

def predict_hmm_regime(features_oos, winsor_bounds, scaler, hmm, pca, bull_st,
thr):
    X_pca   = transform_robust_pca(features_oos, winsor_bounds, scaler, pca)
    probs   = hmm.predict_proba(X_pca)[:, bull_st]
    return probs, probs > thr

# DIAGNOSTICO OBRIGATORIO v10.8:
# 1. Comparar PC1 com/sem winsorização em Março 2020 e Nov 2022:
#    Com StandardScaler: PC1[crash] >> 5 sigma (autovetor contaminado)
#    Com RobustScaler + Winsor: PC1[crash] controlado, dentro de 3 sigma
# 2. Verificar que var_exp >= 80% em todas as janelas expandidas
# 3. Correlação de PC1 robusto com vix_crypto ou realized_vol_30d
#    (interpretabilidade confirma que PC1 captura estrutura real, nao crash)
```

4.4 Comparativo: StandardScaler vs. RobustScaler + Winsorização

Aspecto	v10.7 (StandardScaler + PCA)	v10.8 (Winsorização + RobustScaler + PCA)
Sensibilidade a outliers	ALTA — divide por std global que inclui crashes	MÍNIMA — mediana + IQR imunes a fat tails
PC1 em crash (FTX Nov 2022)	Aponta na direção do crash — contamina HMM	Controlado por Winsorização 99% — estrutura preservada
PC1 em regime normal	Distorcido pelo peso dos crashs históricos	Captura volatilidade estrutural do mercado
Expansão dos bounds	Global (fixo) — não varia com o regime	Por janela de treino expanding — adapta ao histórico disponível
Custo computacional	O(N) por janela	O(N) + Winsorização O(N×k) — desprezível

PARTE 5 — TRIPLE-BARRIER: MARKET IMPACT PELA LEI DA RAIZ QUADRADA (REVISADO v10.10)

5 Triple-Barrier: Detecção HLC, Slippage √(ordem/volume), P&L Microestrutural Real

CORREÇÃO v10.10: Market Impact Linear → Lei da Raiz Quadrada

v10.9 usou exit_price = barrier_sl − pavio × impact_factor (linear). Modelos Kyle (1985) e Almgren-Chriss (2001) estabelecem: impacto de mercado ∝ √(ordem/volume). Liquidez esgota de forma sub-linear: dobrar o tamanho da ordem não dobra o slippage, mas multiplicá-lo por √2 sim. Em flash crash (volume[t] colapsado), √(order_size/volume[t]) cresce rapidamente — capturando o custo real de forçar capital através de funil de liquidez estreito.

5.1 Fundamento: Por que a Lei da Raiz Quadrada

Da microestrutura à fórmula aplicada

Kyle (1985): λ = impacto permanente por unidade de ordem. Preço move λ×Q onde Q é o volume da ordem.

Almgren-Chriss (2001): custo de execução = σ × √(Q/V) onde V é o volume de mercado disponível.

Empírico: meta-análise de Bouchaud et al. (2010) em 12+ mercados confirma: ΔP ∝ σ × √(Q/V).

Em flash crash: V colapsa (market makers retiram quotes), Q/V explode → slippage se torna severo mesmo para ordens pequenas. O fator linear captura isso de forma inadequada.

Interpretação: √(Q/V) é a 'participação de mercado fracionária'. Para Q/V = 1% → fator 0.10; para Q/V = 10% → fator 0.316 (não 0.10 × 10 = 1.0 como seria linear).

5.2 Implementação: apply_triple_barrier_sqrt_impact (v10.10)

```
import numpy as np, pandas as pd

def compute_intraday_vol(high_prices: pd.Series, low_prices: pd.Series,
                         ewm_span: int = 20) -> pd.Series:
    """
    Volatilidade intradiária estimada como (High - Low) / Close (Parkinson).
    Proxy para sigma_intraday sem dados tick.
    """
    hl_range = (high_prices - low_prices) / ((high_prices + low_prices) / 2.0)
    return hl_range.ewm(span=ewm_span, min_periods=5).mean()

def compute_sqrt_market_impact(order_size_usdt: float,
                               volume_usdt:     float,
```

```
                                sigma_intraday:  float,
                                eta:             float = 0.1) -> float:
    """
    Slippage pela Lei da Raiz Quadrada do Impacto de Mercado.

    ΔP = eta × sigma_intraday × sqrt(order_size_usdt / volume_usdt)

    order_size_usdt: tamanho da posicao em USDT (calculado pelo sizing)
    volume_usdt:     volume negociado no bar em USDT (Close × Volume)
    sigma_intraday:  volatilidade intradiaria (High-Low / Mid normalizado)
    eta:             parametro de proporcionalidade (calibrar empiricamente)
                     literatura: eta ∈ [0.05, 0.15]. Padrao: 0.10.

    Retorna: fracao de slippage (preco / preco_entrada - 1).
    Cap: max 0.50 (slippage nao pode exceder 50% do preco).
    """
    participation = order_size_usdt / max(volume_usdt, 1e-10)
    slippage_frac = eta * sigma_intraday * np.sqrt(participation)
    return float(min(slippage_frac, 0.50))   # cap conservador

def apply_triple_barrier_v1010(
        close_prices     : pd.Series,
        high_prices      : pd.Series,
        low_prices       : pd.Series,
        volume_series    : pd.Series,
        events           : pd.Index,
        sigma_ewma       : pd.Series,
        sigma_intraday   : pd.Series,
        position_sizes   : pd.Series,   # {event_date: usdt_amount}
        k_tp             : float = 1.5,
        k_sl             : float = 1.5,
        max_holding_days : int   = 5,
        eta              : float = 0.10) -> pd.DataFrame:
    """
    Triple-Barrier v10.10: HLC (v10.8) + Market Impact √(Q/V) (v10.10).

    Calculo de exit_price no SL:
    1. barrier_sl tocado (Low[t] <= barrier_sl)
    2. volume_usdt = Close[t] * Volume[t]  (volume do bar em USDT)
    3. sigma_intr = sigma_intraday[t]      (volatilidade intradiaria do bar)
    4. slippage = eta * sigma_intr * sqrt(order_size / volume_usdt)
    5. exit_price = barrier_sl * (1 - slippage)

    Diferenca critica v10.9 vs v10.10:
    v10.9: exit_price = barrier_sl - pavio * impact_factor  (linear, arbitrario)
    v10.10: exit_price = barrier_sl * (1 - eta*sigma*sqrt(Q/V))  (fisicamente
correto)

    position_sizes: Series com USDT alocado por data de evento (para Q).
    Se position_sizes nao disponivel no backtest (bootstrap):
    usar proxy: capital_total * max_position_pct (cenario conservador).
    """
    assert close_prices.index.equals(high_prices.index)
    assert close_prices.index.equals(low_prices.index)
    assert close_prices.index.equals(volume_series.index)

    results = []
```

```
    price_idx = close_prices.index

    for event_date in events:
        if event_date not in price_idx:
            continue
        t0 = price_idx.get_loc(event_date)
        p0 = close_prices.iloc[t0]

        sigma_t = sigma_ewma.get(event_date, np.nan)
        if np.isnan(sigma_t) or sigma_t <= 0:
            continue

        barrier_tp = p0 * (1 + k_tp * sigma_t)
        barrier_sl = p0 * (1 - k_sl * sigma_t)
        t_end      = min(t0 + max_holding_days, len(close_prices) - 1)

        # Tamanho da ordem para este evento
        q_usdt = float(position_sizes.get(event_date, p0 * 1000))

        label       = 0
        t_touch     = price_idx[t_end]
        exit_price  = close_prices.iloc[t_end]
        slippage_frac = 0.0

        for t in range(t0 + 1, t_end + 1):
            high_t  = high_prices.iloc[t]
            low_t   = low_prices.iloc[t]
            close_t = close_prices.iloc[t]
            vol_t   = volume_series.iloc[t]

            sl_touched = (low_t  <= barrier_sl)
            tp_touched = (high_t >= barrier_tp)

            if sl_touched:
                # Market Impact √(Q/V) no SL
                vol_usdt   = max(close_t * vol_t, 1e-6)
                sig_intr   = float(sigma_intraday.iloc[t]) if t <
len(sigma_intraday) else sigma_t
                slip_frac  = compute_sqrt_market_impact(q_usdt, vol_usdt,
sig_intr, eta)
                exit_price = barrier_sl * (1.0 - slip_frac)
                slippage_frac = slip_frac
                label         = -1
                t_touch       = price_idx[t]
                break
            elif tp_touched:
                exit_price    = barrier_tp   # TP: limite, sem slippage
                slippage_frac = 0.0
                label         = +1
                t_touch       = price_idx[t]
                break

        pnl_real = (exit_price / p0) - 1.0

        results.append({
            'event_date'    : event_date,
            't_touch'       : t_touch,
```

```
            'label'         : label,
            'barrier_tp'    : round(barrier_tp, 6),
            'barrier_sl'    : round(barrier_sl, 6),
            'exit_price'    : round(exit_price, 6),
            'pnl_real'      : round(pnl_real, 6),
            'slippage_frac' : round(slippage_frac, 6),
            'sigma_at_entry': round(sigma_t, 6),
            'p0'            : round(p0, 6),
        })

    return pd.DataFrame(results).set_index('event_date')

# CALIBRACAO DE eta:
# Literatura: eta ∈ [0.05, 0.15].
# Dados reais: comparar stop teorico vs. preco real de execucao em crashes
historicos.
# LUNA mai/2022: volume colapsou 95% nas primeiras 6h do depegging.
#   sqrt(order_size / (0.05 * normal_volume)) ≈ sqrt(20x) ≈ 4.5x mais slippage
# Usar eta=0.10 como ponto de partida. Ajustar com dados reais de execucao se
disponiveis.
# NUNCA usar eta < 0.05 (subestima crash environments de forma perigosa).
```

5.3 Comparativo: Slippage v10.9 (linear) vs. v10.10 (√)

Cenário	Volume relativo (Q/V)	Slippage v10.9 (linear)	Slippage v10.10 (√ impact)	Diferença
Mercado normal	Q/V = 0.5%	impact_factor × pavio	eta × σ × √0.005 = 0.10 × σ × 0.07	Similar — mercado líquid
Stress moderado	Q/V = 5%	impacto subestimado	eta × σ × √0.05 = 0.10 × σ × 0.22	√ maior — captura reduçã de liquidez
Flash crash (LUNA)	Q/V = 20% (volume colapsado)	impacto linear = 4× base	eta × σ × √0.20 = 0.10 × σ × 0.45	√ mais conservador qu linear aqui
Pânico extremo (FTX)	Q/V = 50% (book esvaziado)	impacto = 10× base (irreal)	eta × σ × √0.50 = 0.10 × σ × 0.71 (cap 50%)	√ fisicamente correto — evita overfit de cauda

PARTE 6 — CFI COM VARIATION OF INFORMATION (REVISADO v10.4)

6 Clustered Feature Importance: Variation of Information

CRÍTICA A — Incorporada v10.4: Correlação Linear é Ingênua para LightGBM

O clustering hierárquico do CFI v10.3 usava distância baseada em correlação de Pearson. LightGBM explora relações não-lineares. Duas features com correlação zero podem ter Mutual Information alta — são redundantes para o modelo mas o clustering linear as trata como independentes. Variation of Information (VI) detecta redundância de qualquer tipo: linear, não-linear, threshold, polinomial. É o único framework com garantia matemática de consistência (VI é métrica própria — satisfaz desigualdade triangular). Referência: Lopez de Prado, AFML 2018, Cap. 3.

6.1 Fundamento Matemático da VI

Mutual Information e Variation of Information

Mutual Information: I(X;Y) = H(X) + H(Y) - H(X,Y)

I(X;Y) captura toda dependência informacional entre X e Y — linear E não-linear.

Variation of Information: VI(X,Y) = H(X|Y) + H(Y|X) = H(X,Y) - I(X,Y)

VI = 0 → X e Y são funcionalmente idênticas (redundância total). VI alta → informativamente independentes.

Pearson = 0 NÃO implica independência informacional. VI = 0 implica. Esta é a diferença que importa para o LightGBM.

6.2 Implementação da VI e Clustering

```
import numpy as np, pandas as pd
from sklearn.preprocessing import KBinsDiscretizer

def variation_of_information(x: np.ndarray, y: np.ndarray, n_bins: int = 10) ->
float:
    """
    VI(X,Y) normalizada em [0,1].
    0 = features informativamente idênticas | 1 = informativamente independentes.
    n_bins adaptativo: min(10, sqrt(N)) — reduz sensibilidade com N pequeno.
    Referência: Lopez de Prado, AFML 2018, Cap. 3.
    """
    n = len(x)
    bins = max(5, min(n_bins, int(np.sqrt(n))))
    disc = KBinsDiscretizer(n_bins=bins, encode='ordinal', strategy='quantile')
```

```
    xd = disc.fit_transform(x.reshape(-1,1)).ravel().astype(int)
    yd = disc.fit_transform(y.reshape(-1,1)).ravel().astype(int)

    def entropy(z):
        _, cnts = np.unique(z, return_counts=True)
        p = cnts / cnts.sum()
        return -np.sum(p * np.log2(p + 1e-12))

    hx  = entropy(xd)
    hy  = entropy(yd)
    hxy = entropy(xd * (bins+1) + yd)   # joint via pairs
    mi  = hx + hy - hxy
    vi  = hx + hy - 2*mi
    return max(vi / max(hxy, 1e-12), 0.0)  # normalizada, não negativa

def compute_vi_distance_matrix(feature_df: pd.DataFrame, n_bins: int = 10) ->
pd.DataFrame:
    """Matriz de distância VI para todas as features. O(n_features^2)."""
    feats = feature_df.columns.tolist(); n = len(feats)
    D = np.zeros((n,n))
    for i in range(n):
        for j in range(i+1,n):
            v = variation_of_information(feature_df[feats[i]].values,
                                         feature_df[feats[j]].values, n_bins)
            D[i,j]=D[j,i]=v
    return pd.DataFrame(D, index=feats, columns=feats)

def clustered_feature_importance_vi(X_train, y_train, X_val, y_val,
                                    feature_names, vi_threshold=0.30,
                                    n_bins=10, n_repeats=30):
    """
    CFI com VI como métrica de distância. Cinco passos:
    1. Treinar RF base
    2. Calcular matriz VI e clusterizar (ward linkage)
    3. Importância por CLUSTER (permutação conjunta)
    4. Importância within-cluster (permutação individual)
    5. Importância final = cluster_imp × within_cluster_imp

    vi_threshold: VI máxima para agrupar no mesmo cluster.
    0.30 = features compartilham > 70% da informação mútua.
    Calibrar com heatmap ANTES de executar (seção 6.4).
    """
    from sklearn.ensemble import RandomForestClassifier
    from sklearn.inspection import permutation_importance
    from scipy.cluster.hierarchy import linkage, fcluster
    from scipy.spatial.distance import squareform
    from sklearn.metrics import roc_auc_score

    # Passo 1: RF base
    rf = RandomForestClassifier(n_estimators=500, max_depth=6, min_samples_leaf=20,
                                max_features='sqrt', class_weight='balanced',
random_state=42)
    rf.fit(X_train, y_train)

    # Passo 2: Clustering por VI
    feat_df     = pd.DataFrame(X_train, columns=feature_names)
    dist_mat    = compute_vi_distance_matrix(feat_df, n_bins=n_bins)
```

```
    link        = linkage(squareform(dist_mat.values), method='ward')
    cluster_ids = fcluster(link, t=vi_threshold, criterion='distance')
    clusters    = {}
    for feat, cid in zip(feature_names, cluster_ids):
        clusters.setdefault(int(cid), []).append(feat)

    print(f"VI Clustering (thr={vi_threshold}): {len(clusters)} clusters")
    for cid, feats in sorted(clusters.items()):
        inner = ([dist_mat.loc[a,b] for a in feats for b in feats if a!=b]
                 if len(feats)>1 else [0])
        print(f"  Cluster {cid} (VI_interno_avg={np.mean(inner):.3f}): {feats}")

    # Passo 3: Importância por cluster
    auc_orig = roc_auc_score(y_val, rf.predict_proba(X_val)[:,1])
    clust_imp = {}; rng = np.random.RandomState(42)
    for cid, feats in clusters.items():
        idx = [feature_names.index(f) for f in feats]
        X_p = X_val.copy(); drops = []
        for _ in range(n_repeats):
            X_p[:,idx] = rng.permutation(X_val[:,idx])
            drops.append(auc_orig - roc_auc_score(y_val,
rf.predict_proba(X_p)[:,1]))
            X_p[:,idx] = X_val[:,idx]

clust_imp[cid]={'features':feats,'imp_mean':np.mean(drops),'imp_std':np.std(drops)}

    # Passo 4: Importância within-cluster
    within = {}
    for cid, cinfo in clust_imp.items():
        feats = cinfo['features']
        if len(feats)==1: within[feats[0]]=1.0; continue
        idx    = [feature_names.index(f) for f in feats]
        rf_sub = RandomForestClassifier(n_estimators=200, random_state=42)
        rf_sub.fit(X_train[:,idx], y_train)
        perm   = permutation_importance(rf_sub, X_val[:,idx], y_val, n_repeats=20,
random_state=42)
        total  = max(perm.importances_mean.sum(), 1e-10)
        for f, imp in zip(feats, perm.importances_mean): within[f]=max(imp,0)/total

    # Passo 5: Importância final
    final_imp = {}
    for cid, cinfo in clust_imp.items():
        ci = max(cinfo['imp_mean'],0)
        for f in cinfo['features']: final_imp[f]=ci*within.get(f,0)

    return pd.Series(final_imp).sort_values(ascending=False), clusters, clust_imp
```

6.3 Clusters VI Esperados para o SNIPER

Cluster Esperado	Features Prováveis	VI Interno	Insight
MOMENTUM	ret_1d, ret_5d, ret_20d, fracdiff_zscore, fracdiff_accel	< 0.20	Mesma dimensão informacional de momentum em escalas diferentes. Pearson detectaria parcialmente; VI detecta a redundância fracionária adicional.

Cluster Esperado	Features Prováveis	VI Interno	Insight
DERIVATIVOS	funding_rate_ma7d, basis_3m	< 0.25	Sentimento de futuros. Backwardation extrema e funding extremo ocorrem juntos — dependência não-linear não capturada por Pearson.
REGIME MACRO	hmm_prob_bull, stablecoin_chg30, dvol_zscore	0.25–0.35	Capturam regime sistêmico com sobreposição moderada. VI distingue melhor do que correlação.
ATIVO-ESPECÍFICO	vol_ratio, unlock_pressure_rank, market_cap_rank	0.35–0.50	Informações relativamente independentes.

6.4 Heatmap de VI: Obrigatório Antes do Clustering

```
def plot_vi_heatmap(dist_matrix: pd.DataFrame, output_path: str =
"vi_heatmap.png"):
    """
    Gerar ANTES de definir vi_threshold.
    Inspecionar visualmente para determinar threshold adequado.
    Salvar como artefato permanente do backtest.
    """
    import matplotlib.pyplot as plt; import seaborn as sns
    fig, ax = plt.subplots(figsize=(12,10))
    sns.heatmap(dist_matrix, annot=True, fmt=".2f", cmap="YlOrRd_r", vmin=0,
vmax=1, ax=ax)
    ax.set_title("Variation of Information Distance Matrix
"
                 "0=idênticas | 1=independentes", fontsize=12)
    plt.tight_layout(); plt.savefig(output_path, dpi=150)
    print(f"Heatmap salvo: {output_path}. Definir vi_threshold antes de executar
CFI.")
```

6.5 Comparativo: Pearson vs. Variation of Information

Critério	Pearson/Spearman (v10.3)	Variation of Information (v10.4)
Detecta dependência linear	Sim	Sim
Detecta dependência não-linear	NÃO — problema central	Sim — qualquer tipo de dependência
ret_1d vs |ret_1d| (módulo)	Spearman ≈ 0 (parece independente)	VI baixo (são informativamente dependentes — correto)
Propriedade métrica	NÃO em geral	SIM — satisfaz desigualdade triangular
Sensibilidade a n_bins	N/A	Usar n_bins = min(10, sqrt(N))
Referência	Pearson (1895)	Lopez de Prado, AFML 2018, Cap. 3

PARTE 7 — IAs META-RACIOCINADORES + VALIDAÇÃO DE ALPHA NÃO-LINEAR (REVISADO v10.4)

7 IAs Meta-Raciocinadores: Alpha Real vs. Redundância Não-Linear

CRÍTICA C — Incorporada v10.4: OLS Cria Resíduos Espúrios

Quando se valida 'as IAs adicionam informação além das features quant?', usar OLS assume que a IA copia as features linearmente. O LightGBM do meta-modelo aprende interações não-lineares. O resíduo do OLS captura a não-linearidade que o OLS não modelou — e incorretamente rotula isso como alpha puro da IA. Solução: LightGBM raso (max_depth=2) como regressor de ortogonalização. ESCOPO: esta seção é para a fase de PESQUISA (Fase 3). No sistema ao vivo, os scores das IAs entram no meta-modelo normalmente.

7.1 Operação das IAs (inalterada)

REGRA INVIOLÁVEL

Todo dado factual coletado por APIs ANTES da chamada à IA. A IA recebe JSON com números reais — INTERPRETA, não COLETA.

IA 1 — META-BULL (Claude)

- Argumento mais robusto e honesto para BULLISH
- Referenciar EXCLUSIVAMENTE dados do JSON
- Se dados não suportam: declarar score 0

IA 2 — META-BEAR (GPT-4o)

- Argumento mais robusto para NÃO entrada
- Riscos não capturados pelos modelos quant
- Se oportunidade clara: score 0 bearish

```
# Output das IAs (inalterado — entram no meta-modelo como features)
{
  "score"                   : 7.5,   // 0-10
  "confianca"               : 8,     // 0-10
  "argumento_principal"     : "...", // max 150 palavras
  "score_confianca_composta": 6.0    // score * confianca / 10
}
# META_FEATURES inclui: score_ia_bull, score_ia_bear, ias_net_score
# O LightGBM/Logística do meta-modelo integra IA + quant de forma não-linear
```

7.2 Validação de Alpha com Ortogonalização Não-Linear

Quando executar esta validação

Fase 3 do roadmap (pesquisa). Pergunta: as IAs valem o custo de API? Critério de aprovação: alpha_retido_pct > 0.30. Critério de reprovação: alpha_retido_pct < 0.15 OU r2_ortho > 0.80.

```
from lightgbm import LGBMRegressor
from sklearn.metrics import r2_score
import numpy as np, pandas as pd

def validate_ia_alpha_nonlinear(meta_df: pd.DataFrame, quant_features: list,
                                ia_score_col: str = 'ias_net_score',
                                target_col: str = 'return_5d') -> dict:
    """
    Valida se as IAs adicionam alpha além das features quant.
    Ortogonalização NÃO-LINEAR via LightGBM raso (max_depth=2).

    Processo:
    1. LightGBM raso: prever score_ia usando features quant
       Captura interações não-lineares que o OLS perderia
    2. Resíduo = score_ia - LightGBM_pred(quant_features)
       É o que a IA expressou que NÃO pode ser derivado das features quant não-
linearmente
    3. Correlação do resíduo com retorno_5d = alpha genuíno das IAs
    4. alpha_retido_pct = |corr_residuo_NL| / |corr_score_puro|

    OLS também calculado para referência — mostrará alpha inflado (demonstra a
crítica).
    """
    split    = int(len(meta_df)*0.70)
    train_df = meta_df.iloc[:split]
    val_df   = meta_df.iloc[split:]

    X_train_q = train_df[quant_features].values; y_train_s =
train_df[ia_score_col].values
    X_val_q   = val_df[quant_features].values;   y_val_s   =
val_df[ia_score_col].values
    y_val_ret = val_df[target_col].values

    # ── LGBM RASO: ortogonalização não-linear ──────────────────────────
    lgbm_ortho = LGBMRegressor(
        max_depth=2, n_estimators=100, learning_rate=0.05,
        min_child_samples=20, subsample=0.8, random_state=42, verbose=-1
    )
    lgbm_ortho.fit(X_train_q, y_train_s)
    score_ia_pred_nl  = lgbm_ortho.predict(X_val_q)
    score_ia_res_nl   = y_val_s - score_ia_pred_nl
    r2_ortho          = r2_score(y_val_s, score_ia_pred_nl)

    # ── OLS: para demonstrar a crítica (referência) ────────────────────
    from sklearn.linear_model import LinearRegression
    ols = LinearRegression().fit(X_train_q, y_train_s)
    score_ia_res_ols = y_val_s - ols.predict(X_val_q)

    # ── CORRELAÇÕES COM RETORNO ────────────────────────────────────────
    corr_puro       = np.corrcoef(y_val_s,          y_val_ret)[0,1]
```

```
    corr_residuo_nl = np.corrcoef(score_ia_res_nl,  y_val_ret)[0,1]
    corr_residuo_ols= np.corrcoef(score_ia_res_ols, y_val_ret)[0,1]
    alpha_retido    = abs(corr_residuo_nl) / max(abs(corr_puro), 1e-10)

    print(f"=== VALIDAÇÃO ALPHA IAs: {ia_score_col} ===")
    print(f"  R2 LGBM ortho (quanto IA replica quant NL): {r2_ortho:.3f}")
    print(f"  Corr score PURO vs retorno_5d:              {corr_puro:.4f}")
    print(f"  Corr RESÍDUO NL vs retorno_5d (CORRETO):   {corr_residuo_nl:.4f}")
    print(f"  Corr RESÍDUO OLS vs retorno_5d (REFERÊNCIA):{corr_residuo_ols:.4f}")
    print(f"  Alpha retido após ortogonalização NL:       {alpha_retido:.1%}")

    if r2_ortho > 0.70:
        print("  ALERTA: IAs replicam >70% das features quant não-linearmente.")
    if alpha_retido > 0.30:
        print("  APROVADO: IAs têm alpha genuíno. Custo de API justificado.")
    elif alpha_retido < 0.15:
        print("  REPROVADO: IAs sem alpha real. Remover da pipeline.")
    else:
        print("  ZONA AMBÍGUA: decisão manual documentada.")

    if abs(corr_residuo_ols) > abs(corr_residuo_nl)*1.2:
        ratio = abs(corr_residuo_ols)/max(abs(corr_residuo_nl),1e-10)
        print(f"  OLS teria SUPERESTIMADO alpha em {ratio:.1f}x vs LightGBM.")

    return {'r2_ortho':r2_ortho, 'corr_score_puro':corr_puro,
            'corr_residuo_nl':corr_residuo_nl,
'corr_residuo_ols':corr_residuo_ols,
            'alpha_retido_pct':alpha_retido}

# —— POR QUE OLS SUPERESTIMA ————————————————————————————————————————
# Cenário típico: score_ia_bear = 8 quando funding_extremo AND basis_backwardation
# OLS: regride score_bear em funding e basis separadamente — não captura a
interação
# Resíduo OLS fica inflado com a não-linearidade da interação
# LGBM max_depth=2: aprende a interação funding × basis diretamente
# Resíduo LGBM é genuinamente o que a IA sabe além do que os dados quant mostram
```

7.3 Critérios de Aprovação das IAs

Métrica	Critério	Consequência
alpha_retido_pct	> 0.30: aprovado	IAs mantidas. Custo de API justificado.
alpha_retido_pct	0.15–0.30: zona ambígua	Decisão manual documentada. Reavaliar a cada 6 meses.
alpha_retido_pct	< 0.15: reprovado	Remover IAs. Operar sem custo de API.
r2_ortho	> 0.80: alerta	IAs são função NL das features quant. Mesmo com corr residual moderada: custo provavelmente injustificado.

7.4 Reflexão Estruturada

```
CREATE TABLE ia_reflections (
```

```
    id INTEGER PRIMARY KEY, timestamp DATETIME, ativo TEXT, ia_modelo TEXT,
    hipotese TEXT, prob_estimada REAL, resultado_real REAL,
    tipo_erro TEXT, licao TEXT,
    -- Campos adicionados v10.4:
    alpha_retido_pct REAL,   -- alpha retido após ortogonalização NL no período
    r2_ortho_at_time REAL    -- R2 do LGBM ortho no momento do trade
);
```

PARTE 8 — META-LABELING: STACKING ORTOGONAL + UNIQUENESS DINÂMICA (REVISADO v10.10)

8 Meta-Labeling: P_bma via Purged K-Fold, Uniqueness Real, Pesos Assimétricos, CPCV

PROBLEMA CORRIGIDO v10.7: Stacking Não-Ortogonal

O meta-modelo usa P_bma (predição do modelo primário) como feature de entrada. Se P_bma é gerado por walk-forward simples e o CPCV do meta-modelo embaralha blocos temporais, P_bma[t] pode ter sido gerado por um modelo que viu o período correspondente ao fold de teste do CPCV. O meta-modelo aprenderia a assinatura temporal das predições do primário — vazamento de segunda ordem. Correção: gerar P_bma via Purged K-Fold sobre TODO o dataset antes do meta-modelo. Cada P_bma[t] gerado por um fold que nunca viu t.

8.1 Stacking Ortogonal: P_bma via Purged K-Fold

Por que walk-forward simples não é suficiente para stacking

Walk-forward gera P_bma[t] treinando o primário com dados até t−1. Parece correto.

Problema: o CPCV do meta-modelo cria folds que EMBARALHAM o dataset. O fold de teste pode incluir datas próximas a datas de treino do modelo primário.

Consequência: P_bma para o período de teste do CPCV foi gerado por um primário que 'viu' dados vizinhos a esse período. A assinatura temporal de P_bma vaza informação sobre quando a predição foi feita.

Purged K-Fold sobre todo o dataset: P_bma[t] gerado pelo fold que exclui a janela [t−embargo, t+embargo]. Cada observação nunca viu a si mesma. Folds alinhados e expurgados. O meta-modelo recebe predições genuinamente OOS.

Referência: Lopez de Prado, AFML 2018, Cap. 7 (Stacking em séries temporais financeiras).

8.2 Geração Ortogonal de P_bma via Purged K-Fold

```
import numpy as np, pandas as pd
from sklearn.model_selection import KFold
from lightgbm import LGBMClassifier
from sklearn.linear_model import LogisticRegression

def generate_pbma_purged_kfold(feature_df: pd.DataFrame,
                               target_series: pd.Series,
                               sample_weights: pd.Series,
                               primary_models: dict,
                               n_splits: int = 10,
                               embargo_pct: float = 0.01) -> pd.Series:
```

```
    """
    Gera P_bma de forma ortogonal via Purged K-Fold sobre todo o dataset.

    Para stacking correto: P_bma[t] nunca foi gerado por um modelo que viu t.
    Cada P_bma[t] e gerado pelo fold k que exclui a janela embargada em torno de
t.

    n_splits=10: 10 folds. Mais splits = mais dados de treino por fold.
    embargo_pct=0.01: 1% das obs eliminadas ao redor de cada fronteira de fold.

    primary_models: {'gbm': LGBMClassifier, 'rf': RandomForestClassifier}
    Pesos BMA por accuracy OOS rolling 30d (calculados dentro de cada fold).

    Retorna: pd.Series com P_bma para cada observacao -- genuinamente OOS.
    """
    n        = len(feature_df)
    p_bma    = pd.Series(np.nan, index=feature_df.index)
    embargo  = max(1, int(n * embargo_pct))

    kf = KFold(n_splits=n_splits, shuffle=False)

    for fold_idx, (train_idx, test_idx) in enumerate(kf.split(feature_df)):
        # Purge: remover observacoes de treino proximas as fronteiras do fold de
teste
        test_min, test_max = test_idx.min(), test_idx.max()
        train_purged = train_idx[
            ~((train_idx >= test_min - embargo) & (train_idx <= test_max +
embargo))
        ]
        if len(train_purged) < 60:
            continue

        X_tr = feature_df.iloc[train_purged].values
        y_tr = target_series.iloc[train_purged].values
        w_tr = sample_weights.iloc[train_purged].values
        X_te = feature_df.iloc[test_idx].values

        # Treinar cada modelo base e calcular P_oos para o fold
        fold_preds = {}
        for name, model_cls in primary_models.items():
            m = model_cls()
            m.fit(X_tr, y_tr, sample_weight=w_tr)
            fold_preds[name] = m.predict_proba(X_te)[:, 1]

        # BMA: pesos por accuracy no treino deste fold (proxy conservador)
        accs = {}
        for name, preds_tr in {n: m.predict_proba(X_tr)[:,1] for n,m in
                               {k: v() for k,v in
primary_models.items()}.items()}.items():
            accs[name] = ((preds_tr > 0.5) == y_tr).mean()

        total_acc = max(sum(accs.values()), 1e-10)
        w_bma     = {k: v / total_acc for k, v in accs.items()}

        # P_bma ponderado para este fold
        p_fold = sum(w_bma[k] * fold_preds[k] for k in fold_preds)
        p_bma.iloc[test_idx] = p_fold
```

```
        print(f"  Fold {fold_idx+1}/{n_splits}: "
              f"N_train={len(train_purged)}, N_test={len(test_idx)}, "
              f"embargo={embargo}")

    # Preencher NaN restantes (primeiros/ultimos obs sem fold completo)
    p_bma = p_bma.ffill().bfill()
    return p_bma

# PIPELINE CORRETO DE STACKING ORTOGONAL (v10.7):
# 1. Calcular fracdiff_zscore com expanding window + log-espaco (v10.7)
# 2. Calcular sigma_ewma e Triple-Barrier labels (v10.5)
# 3. GERAR P_bma via Purged K-Fold sobre TODO o dataset (v10.7 -- NOVO)
# 4. Concatenar P_bma com features do meta-modelo
# 5. Treinar meta-modelo com CPCV usando P_bma ortogonal como feature
#
# ERRADO (v10.6 e anteriores):
# 1. Walk-forward do primario --> P_bma_wf
# 2. CPCV do meta-modelo com P_bma_wf como feature --> vazamento de segunda ordem
#
# CORRETO (v10.7):
# 1. Purged K-Fold do primario sobre TODO dataset --> P_bma_pkf (cada ponto e OOS)
# 2. CPCV do meta-modelo com P_bma_pkf como feature --> stacking ortogonal
```

8.3 Label Uniqueness Dinâmica com t_touch Real (v10.6 — mantido)

```
def compute_label_uniqueness_dynamic(triple_barrier_df: pd.DataFrame) ->
pd.Series:
    """Unicidade usando duração real do trade via t_touch (v10.6). Sem
mudancas."""
    t0 = pd.to_datetime(triple_barrier_df.index).values.astype(np.int64)
    t1 = pd.to_datetime(triple_barrier_df['t_touch']).values.astype(np.int64)
    overlap     = (t0[None,:] <= t1[:,None]) & (t1[None,:] >= t0[:,None])
    concurrency = overlap.sum(axis=1).astype(float)
    return pd.Series(1.0/np.maximum(concurrency,1.0),
index=triple_barrier_df.index)

def calculate_effective_N_dynamic(triple_barrier_df):
    u=compute_label_uniqueness_dynamic(triple_barrier_df); N=u.sum()
    if N<60: m="LOGISTIC_ONLY"
    elif N<120: m="LGBM_STRICT"
    else: m="LGBM_STANDARD"
    return N,u,m
```

8.4 Pesos Assimétricos: Penalidade de Stop-Loss (v10.6 — mantido)

```
def compute_meta_sample_weights(triple_barrier_df, uniqueness,
                                halflife_days=180, sl_penalty=2.0):
    """
    Pesos multiplicativos: uniqueness_dinamica x decay_temporal x penalidade_SL.
    sl_penalty=2.0: SL pesa 2x mais que TS. Definir ANTES de ver validacao.
    """
```

Interno


```
    df=triple_barrier_df.copy(); df.index=pd.to_datetime(df.index)
    w_uniq  =uniqueness.reindex(df.index).fillna(uniqueness.mean()).values
    days_ago=(df.index.max()-df.index).days
    w_time  =np.exp(-days_ago/halflife_days)
    w_sl    =df['label'].map({1:1.0,0:1.0,-
1:float(sl_penalty)}).fillna(1.0).values
    combined=w_uniq*w_time*w_sl
    return pd.Series(combined/combined.sum()*len(combined),index=df.index)
```

8.5 CPCV com Stacking Ortogonal

```
from itertools import combinations
def cpcv_meta_labeling_v107(meta_df, n_splits=6, n_test_splits=2,
                            embargo_pct=0.01, sl_penalty=2.0):
    """
    CPCV com P_bma ortogonal (v10.7).
    meta_df deve conter p_bma_pkf (gerado via Purged K-Fold, nao walk-forward).
    """
    meta_df=meta_df.sort_values('date').reset_index(drop=True)
    n=len(meta_df); splits=np.array_split(np.arange(n),n_splits); results=[]
    for combo in combinations(range(n_splits),n_test_splits):
        test_idx=np.concatenate([splits[i] for i in combo])
        train_idx=np.array([j for j in range(n) if j not in set(test_idx)])
        embargo=max(1,int(n*embargo_pct))
        train_idx=train_idx[~((train_idx>=test_idx.min()-
embargo)&(train_idx<=test_idx.max()+embargo))]
        if len(train_idx)<40 or len(test_idx)<15: continue
        N_eff,uniq,mt=calculate_effective_N_dynamic(meta_df.loc[train_idx])

sw=compute_meta_sample_weights(meta_df.loc[train_idx],uniq,sl_penalty=sl_penalty)
        meta_m=train_meta_model_by_type(meta_df.loc[train_idx,META_FEATURES],
                                        meta_df.loc[train_idx,'Y_meta'],sw,mt)
        p_oos=meta_m.predict_proba(meta_df.loc[test_idx,META_FEATURES])[:,1]

results.append({'auc':roc_auc_score(meta_df.loc[test_idx,'Y_meta'],p_oos)})
    df=pd.DataFrame(results)
    print(f"AUC: {df['auc'].mean():.3f}+-{df['auc'].std():.3f} |
PBO={(df['auc']<0.50).mean():.1%}")
    return df

# META_FEATURES v10.7 inclui 'p_bma_pkf' (gerado via Purged K-Fold)
# nao mais 'p_bma_wf' (walk-forward simples)
META_FEATURES_V107 = [
    'p_bma_pkf',         # NOVO v10.7: P_bma ortogonal via Purged K-Fold
    'score_ia_bull',     # score IA bullish (se IAs aprovadas)
    'score_ia_bear',     # score IA bearish
    'ias_net_score',     # score liquido das IAs
    'hmm_prob_bull',     # probabilidade de regime bull (via PCA+HMM v10.7)
    'sigma_ewma',        # volatilidade EWMA no momento do sinal
    'funding_rate_ma7d', # funding atual
    'basis_3m',          # basis do mercado futuro
    'stablecoin_chg30',  # macro regime
    'unlock_pressure_rank', # pressao de unlock
]
```


Interno

8.6 Calibração Isotônica Pooled com Time-Decay por Cluster VI (v10.10)

Overfitting Isotônico em N Pequeno — Pooling por Cluster VI

Dois problemas: (1) Tree-based models produzem scores comprimidos em direção a 0.5 — calibração isotônica corrige. (2) Isotonic Regression não-paramétrica com N_eff < 60 por ativo cria step functions erráticas: scores 0.81 e 0.82 resultam em alocações radicalmente diferentes. Correção v10.9: pooling OOS de todos os ativos do mesmo cluster VI (comportamento microestrutural similar). Calibrador treinado em 1.500+ decisões conjuntas → curva suave, monotônica e generalizável.

```
from sklearn.isotonic import IsotonicRegression
import numpy as np, pandas as pd

# ──── POOLING POR CLUSTER VI ──────────────────────────────────────────────
# Clusters definidos pelo heatmap de Variation of Information (Parte 6).
# Ativos com VI similar compartilham comportamento microestrutural.
# Exemplo de clusters:
#   Cluster A (BTC-like, large cap): BTC, ETH, BNB
#   Cluster B (mid-cap momentum):    SOL, AVAX, MATIC, ARB
#   Cluster C (small-cap high-vol):  INJ, SEI, TIA, PENDLE
VI_CLUSTERS = {
    'cluster_A': ['BTCUSDT','ETHUSDT','BNBUSDT'],
    'cluster_B': ['SOLUSDT','AVAXUSDT','MATICUSDT','ARBUSDT'],
    'cluster_C': ['INJUSDT','SEIUSDT','TIAUSDT'],
}  # Ajustar conforme clusters reais do heatmap VI

def _compute_time_decay_weights(dates: pd.DatetimeIndex,
                                halflife_days: int = 180) -> np.ndarray:
    """
    Pesos de decaimento temporal exponencial.
    Observacoes recentes recebem peso proximo a 1.0.
    Observacoes com halflife_days atras recebem peso 0.5.
    Observacoes com 2 x halflife_days atras recebem peso 0.25.

    Garante que a calibracao isotonica reflita a eficiencia de mercado ATUAL,
    nao a media de 5 anos de microestrutura diferente.

    halflife_days=180: seis meses. Ajustar para mercados mais estaticos.
    halflife_days=90: tres meses. Para crypto em regime de alta volatilidade.
    """
    max_date  = dates.max()
    days_ago  = (max_date - dates).days.astype(float)
    decay_cst = np.log(2) / halflife_days   # lambda para halflife
    weights   = np.exp(-decay_cst * days_ago)
    return weights / weights.sum() * len(weights)   # normalizar: media = 1.0

def calibrate_isotonic_pooled(oos_predictions: dict,
                              vi_clusters: dict,
                              min_pool_size: int = 150) -> dict:
    """
    Calibração isotônica POOLED por cluster VI (v10.9).

    oos_predictions: {ativo: {'p_meta_raw': array, 'y_true': array}}
```

```
    vi_clusters:     {cluster: [lista de ativos]}
    min_pool_size:   N mínimo para calibração. Abaixo disso: usar cluster inteiro.

    Retorna: {cluster: IsotonicRegressor fittado no pool conjunto}
    Cada ativo recebe o calibrador do seu cluster.

    Por que pooling funciona:
    - Ativos no mesmo cluster VI têm distribuição de scores similar
    - Pooling junta N_eff de todos os ativos do cluster: 150-500+ observações
    - Curva isotônica é suave e generalizável — não step function errática
    """
    cluster_calibrators = {}

    for cluster_name, ativos in vi_clusters.items():
        # Juntar predições OOS de todos os ativos do cluster
        pool_scores = []
        pool_labels = []

        pool_dates = []
        for ativo in ativos:
            if ativo not in oos_predictions:
                continue
            pool_scores.extend(oos_predictions[ativo]['p_meta_raw'].tolist())
            pool_labels.extend(oos_predictions[ativo]['y_true'].tolist())
            pool_dates.extend(oos_predictions[ativo]['dates'].tolist())  # datas
das predicoes OOS

        pool_scores = np.array(pool_scores)
        pool_labels = np.array(pool_labels)
        pool_dates  = pd.to_datetime(pool_dates)

        if len(pool_scores) < min_pool_size:
            print(f"AVISO: {cluster_name} tem apenas {len(pool_scores)} obs. "
                  f"Calibracao pode ser instavel. Considerar merger de clusters.")

        # Fit Isotonic Regression no pool conjunto com TIME-DECAY (v10.10)
        # IsotonicRegression aceita sample_weight -- recentes pesam mais.
        # halflife_days=180: peso cai para 50% em 6 meses, para ~5% em 2 anos.
        pool_weights = _compute_time_decay_weights(pool_dates, halflife_days=180)
        iso = IsotonicRegression(out_of_bounds='clip')
        iso.fit(pool_scores, pool_labels, sample_weight=pool_weights)

        # Calcular ECE do pool
        ece = _compute_ece(pool_scores, pool_labels)
        p_cal = iso.predict(pool_scores)
        ece_after = _compute_ece(p_cal, pool_labels)

        print(f"{cluster_name} (N={len(pool_scores)}): "
              f"ECE {ece:.4f} → {ece_after:.4f} {'OK' if ece_after<0.05 else
'AVISO'}")

        cluster_calibrators[cluster_name] = {
            'iso'       : iso,
            'n_pool'    : len(pool_scores),
            'ece_before': round(ece, 4),
            'ece_after' : round(ece_after, 4),
            'ativos'    : ativos,
```

```
        }

    return cluster_calibrators

def _compute_ece(probs: np.ndarray, labels: np.ndarray, n_bins: int = 10) ->
float:
    """Expected Calibration Error."""
    bin_edges = np.linspace(0, 1, n_bins + 1)
    ece = 0.0
    for i in range(n_bins):
        mask = (probs >= bin_edges[i]) & (probs < bin_edges[i+1])
        if mask.sum() == 0: continue
        ece += (mask.sum()/len(probs)) * abs(probs[mask].mean() -
labels[mask].mean())
    return ece

def apply_calibration_pooled(p_raw: float, ativo: str,
                             cluster_calibrators: dict,
                             vi_clusters: dict) -> float:
    """
    Calibra score bruto usando o calibrador do cluster VI do ativo.
    Chamar ANTES de pmeta_scaling_factor() em producao.
    """
    for cluster_name, ativos in vi_clusters.items():
        if ativo in ativos:
            iso = cluster_calibrators[cluster_name]['iso']
            return float(iso.predict(np.array([p_raw]))[0])
    # Fallback: retornar p_raw se ativo nao encontrado em nenhum cluster
    print(f"AVISO: {ativo} nao encontrado em nenhum cluster VI. Usando p_raw sem
calibracao.")
    return p_raw

# PIPELINE CORRETO v10.10 (producao):
# 1. CPCV gera p_meta_raw por ativo
# 2. Pooling por cluster VI: calibrate_isotonic_pooled(oos_preds, vi_clusters)
# 3. Em producao: p_cal = apply_calibration_pooled(p_raw, ativo, calibrators,
clusters)
# 4. Sizing: phi_z = pmeta_scaling_factor(p_cal)  -- score calibrado no pool
# 5. CVaR usa pnl_real (com slippage v10.9)

# DIFERENCA CRITICA v10.8 → v10.9 → v10.10:
# v10.8: iso.fit(p_meta_oos_ativo, y_true_ativo)
#        N_eff < 60 por ativo = step function erratica
# v10.9: iso.fit(pool_scores, pool_labels)
#        N = 150-500+ = curva suave, mas ignora decaimento temporal
# v10.10: iso.fit(pool_scores, pool_labels, sample_weight=time_decay_weights)
#         N = 150-500+ E peso exponencial temporal = curva suave + relevancia
atual
#         score 0.85 em 2021 pesa ~5% do que score 0.85 em 2025 (halflife=180d)
```

PARTE 9 — CONCEPT DRIFT: C2ST COM DOWNSAMPLING E PR-AUC (REVISADO v10.10)

9 Concept Drift: C2ST Balanceado + KS Diagnóstico

REVISÃO v10.5 + CORREÇÃO v10.6: C2ST Balanceado e Block Bootstrap

v10.5 corrigiu o desbalanceamento 4.5:1 com downsampling. v10.6 corrige o mecanismo do downsampling: rng.choice aleatório sobre o OOS criava dataset de referência sem autocorrelação — RF aprendia a separar série contínua (produção) de série embaralhada (OOS sorteado), não drift real. Resultado: falsos positivos estruturais. Block Bootstrap v10.6: blocos contíguos de 5 dias amostrados do OOS preservam a assinatura de autocorrelação. Ambos os conjuntos (referência e produção) mantêm estrutura temporal — RF caça discrepâncias distributivas reais.

9.1 Arquitetura Dual (inalterada)

Detector	Papel	O que Detecta
C2ST (PRIMÁRIO)	ALARME: drift na distribuição conjunta	Quebra na covariância entre features — detecta antes do KS
KS por cluster VI (SECUNDÁRIO)	DIAGNÓSTICO: qual cluster derivou	Mudança nas distribuições marginais por cluster

9.2 Por que ROC-AUC Sozinho Falha com Desbalanceamento

Cenário	Proporção	ROC-AUC	PR-AUC	Diagnóstico
C2ST v10.4 SEM downsampling	4.5:1 OOS:Prod	0.72 parece ótimo	0.28 péssimo	RF classifica quase tudo como OOS. Drift REAL invisível. Falso negativo sistemático.
C2ST v10.5 COM downsampling	1:1 balanceado	0.72	0.74 consistente	Classes balanceadas. PR-AUC reflete precision real na classe de produção.
Sem drift — distribuições iguais	1:1	0.50	0.50	RF não separa. Correto.

9.2b Block Bootstrap v10.6 vs. Random Sampling v10.5

Método	Estrutura temporal OOS	O que RF aprende	Resultado
Random sampling v10.5	DESTRUÍDA — índices sorteados sem reposição	Separar série contínua (produção) de série embaralhada (OOS sorteado)	Falsos positivos estruturais:

Método	Estrutura temporal OOS	O que RF aprende	Resultado
			drift aparente mesmo sem drift real
Block Bootstrap v10.6	PRESERVADA — blocos contíguos de 5 dias	Separar distribuições com autocorrelação similar — detecta drift real	Falsos positivos reduzidos: RF precisa de discrepância distributiva genuína

9.3 C2ST com Block Bootstrap — Implementação Corrigida

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import roc_auc_score, average_precision_score, brier_score_loss
from scipy.stats import binomtest
import numpy as np, pandas as pd

class C2STDriftDetector:
    # n_bootstrap=50 repeticoes de downsampling para estabilizar estimativas.
    # Em cada repeticao: amostrar n_prod observacoes do OOS aleatoriamente.
    # Classes sempre balanceadas 50/50 dentro de cada repeticao.
    # Drift confirmado somente quando AMBOS ROC-AUC e PR-AUC superam thresholds.
    # Referencia: Lopez-Paz e Oquab (2017), Revisiting Classifier Two-Sample Tests.
    def __init__(self, reference_window_days=90, production_window_days=20,
                 roc_auc_threshold=0.55, pr_auc_threshold=0.60,
                 min_samples_class=20, n_estimators=100, max_depth=3,
                 n_bootstrap=50, block_size=5):
        # block_size=5: blocos de 5 dias contiguos para Block Bootstrap (v10.6)
        # Preserva autocorrelacao intradiaria e interdiaria da serie de referencia.
        # Random sampling (v10.5) criava dataset de referencia sem autocorrelacao --
        # RF separava serie continua de serie embaralhada, nao drift real.
        self.ref_window=reference_window_days;
self.prod_window=production_window_days
        self.roc_thr=roc_auc_threshold; self.pr_thr=pr_auc_threshold
        self.min_samples=min_samples_class; self.n_est=n_estimators
        self.depth=max_depth; self.n_bootstrap=n_bootstrap
        self.block_size=block_size   # Block Bootstrap v10.6

    def _block_sample(self, ref_data: 'pd.DataFrame', n_target: int,
                      block_size: int, rng: 'np.random.RandomState') ->
'pd.DataFrame':
        """
        Block Bootstrap: amostrar blocos contiguos de block_size dias do OOS.
        Preserva autocorrelacao da serie -- impede que RF aprenda a separar
        'serie continua' de 'serie embaralhada' (falso positivo estrutural v10.5).
        """
        n = len(ref_data); blocks = []; total = 0
        while total < n_target:
            start = rng.randint(0, max(1, n - block_size))
            block = ref_data.iloc[start:start+block_size]
```

```
            blocks.append(block); total += len(block)
        sampled = pd.concat(blocks).iloc[:n_target]
        return sampled.reset_index(drop=True)

    def run_check(self, feature_store_history, reference_end_date, feature_names):
        ref_start=reference_end_date-pd.Timedelta(days=self.ref_window)
        prod_end=reference_end_date+pd.Timedelta(days=self.prod_window)

ref_data=feature_store_history.loc[ref_start:reference_end_date,feature_names].dropna
()

prod_data=feature_store_history.loc[reference_end_date:prod_end,feature_names].dropna
()
        n_prod=len(prod_data); n_ref=len(ref_data)
        if n_prod<self.min_samples or n_ref<self.min_samples:
            return {'drift_detected':False,'reason':f'N insuf:
ref={n_ref},prod={n_prod}'}
        roc_aucs,pr_aucs,briers=[],[],[]
        rng=np.random.RandomState(42)
        for seed in range(self.n_bootstrap):
            # BLOCK BOOTSTRAP v10.6: amostrar blocos contiguos do OOS
            # (nao indices aleatorios) para preservar autocorrelacao da serie
            ref_s=self._block_sample(ref_data, n_target=n_prod,
                                     block_size=self.block_size, rng=rng)
            X=np.vstack([ref_s.values,prod_data.values])
            y=np.array([0]*n_prod+[1]*n_prod)
            clf=RandomForestClassifier(n_estimators=self.n_est,max_depth=self.depth,

min_samples_leaf=max(3,int(len(X)*0.025)),class_weight='balanced',random_state=seed)
            skf=StratifiedKFold(n_splits=5,shuffle=True,random_state=seed)
            fr,fp,fb=[],[],[]
            for tr,te in skf.split(X,y):
                clf.fit(X[tr],y[tr]); probs=clf.predict_proba(X[te])[:,1]
                fr.append(roc_auc_score(y[te],probs))
                fp.append(average_precision_score(y[te],probs))
                fb.append(brier_score_loss(y[te],probs))
            roc_aucs.append(np.mean(fr)); pr_aucs.append(np.mean(fp));
briers.append(np.mean(fb))
        roc_mean=np.mean(roc_aucs); pr_mean=np.mean(pr_aucs); br_mean=np.mean(briers)

p_val=binomtest(int(roc_mean*2*n_prod),2*n_prod,p=0.5,alternative='greater').pvalue
        roc_ok=(roc_mean>self.roc_thr)and(p_val<0.05); pr_ok=pr_mean>self.pr_thr
        drift_detected=roc_ok and pr_ok
        ref_full=ref_data.sample(n=n_prod,random_state=42)

clf_full=RandomForestClassifier(n_estimators=self.n_est,max_depth=self.depth,random_s
tate=42)

clf_full.fit(np.vstack([ref_full.values,prod_data.values]),np.array([0]*n_prod+[1]*n_
prod))
        feat_imp=pd.Series(clf_full.feature_importances_,index=feature_names)
        return {
            'drift_detected'      : drift_detected,
            'roc_auc_mean'        : round(roc_mean,4),
            'pr_auc_mean'         : round(pr_mean,4),
            'brier_score'         : round(br_mean,4),
            'p_value_roc'         : round(p_val,6),
            'n_prod'              : n_prod,
            'n_ref_available'     : n_ref,
```

```
            'n_downsampled_oos'   : n_prod,
            'top_drifted_features': feat_imp.nlargest(3).index.tolist(),
            'roc_ok': roc_ok, 'pr_ok': pr_ok, 'brier_ok': br_mean<0.20,
        }

    def pipeline_daily(self, fsh, ref_date, feat_names, ks_diag, orch, notif):
        import sqlite3, json
        res=self.run_check(fsh,ref_date,feat_names)
        with sqlite3.connect('sniper_trades.db') as c:
            c.execute("CREATE TABLE IF NOT EXISTS drift_log (id INTEGER PRIMARY KEY
AUTOINCREMENT,date TEXT,test_type TEXT,details TEXT)")
            c.execute("INSERT INTO drift_log VALUES
(NULL,?,?,?)",(str(ref_date.date()),'C2ST',json.dumps(res)))
        if not res['drift_detected']: return res
        rd=fsh.loc[ref_date-pd.Timedelta(days=self.ref_window):ref_date,feat_names]
        pd_=fsh.loc[ref_date:ref_date+pd.Timedelta(days=self.prod_window),feat_names]
        ks_res=ks_diag.diagnose(rd,pd_)
        roc=res['roc_auc_mean']; pr=res['pr_auc_mean']
        sev="AMBIGUOUS" if pr<self.pr_thr else ("MILD" if roc<0.60 else "MODERATE" if
roc<0.70 else "SEVERE")
        data={**res,'ks_clusters':str(ks_res['drifted_clusters']),'severity':sev}
        if sev=="MILD": notif.send('REGIME_CHANGE','MERCADO',data)
        elif sev=="MODERATE": orch.block_new_entries();
notif.send('REGIME_CHANGE','MERCADO',data)
        elif sev=="SEVERE": orch.halt_all("C2ST_SEVERE");
notif.send('HALT','MERCADO',data)
        return res
```

9.4 KS por Cluster VI — Diagnóstico Secundário

```
class KSClusterDiagnostic:
    def __init__(self,vi_clusters,alpha_global=0.05):
        self.clusters=vi_clusters; self.alpha=alpha_global/max(len(vi_clusters),1)
    def diagnose(self,ref_data,prod_data):
        from scipy.stats import ks_2samp; results={}
        for cid,feats in self.clusters.items():

D,p=ks_2samp(ref_data[feats].mean(axis=1).values,prod_data[feats].mean(axis=1).values
)

results[cid]={'features':feats,'D':round(D,4),'p':round(p,6),'drifted':p<self.alpha}
        return {'clusters':results,'drifted_clusters':[r['features'] for r in
results.values() if r['drifted']]}
```

9.5 Severidade e Ações por ROC-AUC + PR-AUC

ROC-AUC	PR-AUC	Severidade	Ação Imediata
< 0.55	qualquer	NONE	Operação normal. Log silencioso.
0.55–0.70	< 0.60	AMBIGUOUS	Monitoramento 2x/dia. Aguardar 3 dias.
0.55–0.60	≥ 0.60	MILD	Monitoramento 2x/dia. Operação continua.

ROC-AUC	PR-AUC	Severidade	Ação Imediata
0.60–0.70	≥ 0.60	MODERATE	Bloquear NOVAS entradas. Stop loss acelerado. Diagnóstico 7d.
> 0.70	≥ 0.60	SEVERE	HALT TOTAL. Saída gradual todas posições em 24h.

9.6 Retreinamento Após Quebra Estrutural

```
def retrain_with_structural_break(df_old,df_new,break_date,

halflife_old=90,halflife_new=30,embargo_days=10):
    em_s=break_date-pd.Timedelta(days=embargo_days);
em_e=break_date+pd.Timedelta(days=embargo_days)
    old=df_old[df_old.index<em_s].copy(); new=df_new[df_new.index>em_e].copy()
    today=pd.Timestamp.utcnow()
    old['w']=np.exp(-(today-old.index).days/halflife_old)
    new['w']=np.exp(-(today-new.index).days/halflife_new)
    new['w']*=len(old)/max(len(new),1)
    combined=pd.concat([old,new]);
combined['w']/=combined['w'].sum()*len(combined)
    return combined
```

PARTE 10 — POSITION SIZING: KELLY CVaR-LIMITADO + P_META ESCALADO (REVISADO v10.10)

10 Position Sizing: f_base = E[ganho] / CVaR-99% × Φ(z(P_meta))

PROBLEMA CORRIGIDO v10.7: Kelly Plug-in Subestima Tail Risk

Kelly numérico com parâmetros estimados pela distribuição histórica de P&L subestima sistematicamente o risco de cauda em crypto: retornos leptocúrticos (fat tails), saltos de Poisson (crashes repentinos como LUNA, FTX), e liquidez inexistente em gaps extremos. Se o pior cenário do backtest foi −15%, mas o pior cenário real pode ser −40% (gap de liquidez não observado), o Kelly otimizará para sobreviver ao −15% e falhar no −40%. Correção: E[ganho] / CVaR_99% como base do Kelly. Se choques extremos mostram −30%, o numerador é limitado matematicamente.

10.1 Fundamento: f_base via CVaR-99%

Fórmula completa v10.7

CVaR_99% = Expected Shortfall no percentil 1% = média dos 1% piores retornos do P&L histórico da estratégia

f_cvar_base = max(E[P&L], 0) / CVaR_99% (CVaR em valor absoluto)

f_quarter = 0.25 × f_cvar_base (quarter-Kelly conservador sobre base já CVaR-limitada)

Φ(z) = CDF normal em z = (P_meta − 0.5) / sqrt(P_meta × (1 − P_meta))

Tamanho final = f_quarter × Φ(z(P_meta)) × HRP_weight × capital

Se CVaR_99% = 30% (estratégia perde 30% nos 1% piores dias) e E[P&L] = 2%, f_cvar_base = 2%/30% = 6.7%

Quarter-Kelly: 0.25 × 6.7% = 1.67%. Escalado por Φ(z): se P_meta=0.80 → 1.67% × 0.76 = 1.27%

Referência: Lopez de Prado, AFML 2018, Cap. 13 — bet sizing com tail risk.

10.2 Implementação: Kelly CVaR-Limitado

```
from scipy.stats import norm
from scipy.optimize import minimize_scalar
import numpy as np

def compute_cvar(pnl_series: np.ndarray, confidence: float = 0.99) -> float:
    """
    CVaR (Expected Shortfall) no percentil dado.
    CVaR_99% = media dos 1% piores retornos.
```

```
    pnl_series: array de retornos diarios da estrategia no periodo de treino.
    Inclui todos os trades (vencedores e perdedores). Nao apenas os dias operados.
    confidence=0.99: 1% piores dias (tail esquerdo).

    Retorna CVaR como valor POSITIVO (perda esperada em percentual).
    Exemplo: CVaR_99%=0.30 significa que nos 1% piores dias perde-se 30%.
    """
    cutoff = np.percentile(pnl_series, (1 - confidence) * 100)
    tail   = pnl_series[pnl_series <= cutoff]
    if len(tail) == 0:
        return abs(cutoff)
    return abs(tail.mean())   # CVaR como valor positivo

def pmeta_scaling_factor(p_meta: float) -> float:
    """Fator CDF normal para escalar Kelly por P_meta (v10.5 — inalterado)."""
    p = float(np.clip(p_meta, 0.501, 0.999))
    z = (p - 0.5) / np.sqrt(p * (1 - p))
    return float(norm.cdf(z))

def kelly_cvar_limited(pnl_history: np.ndarray,
                       p_meta: float,
                       cvar_confidence: float = 0.99,
                       quarter_kelly_factor: float = 0.25,
                       max_position_pct: float = 0.22,
                       prior_mean: float = None,
                       prior_n: int = 12) -> dict:
    """
    Kelly base limitado por CVaR-99% + scaling por P_meta via CDF normal.

    Passos:
    1. E[P&L] com Shrinkage Bayesiano (v10.5)
    2. CVaR_99% dos piores retornos historicos da estrategia
    3. f_cvar_base = max(E[PnL], 0) / CVaR_99%
    4. f_quarter = 0.25 x f_cvar_base
    5. f_scaled  = f_quarter x Phi(z(P_meta))
    6. Cap: max_position_pct (HRP)

    pnl_history: P&L diario da estrategia nos ultimos N dias de treino.
                 Inclui dias sem trades (retorno = 0) para capturar tail real.
    """
    n        = len(pnl_history)
    mu_sample = pnl_history.mean()

    # Shrinkage Bayesiano da media
    if prior_mean is not None:
        mu_post = (n * mu_sample + prior_n * prior_mean) / (n + prior_n)
    else:
        mu_post = mu_sample

    if mu_post <= 0:
        return {'position_fraction': 0.0, 'reason': 'negative_expectation',
                'e_pnl': mu_post, 'cvar_99': None, 'f_cvar_base': 0.0}

    # CVaR_99%: risco de cauda historico
    cvar_99 = compute_cvar(pnl_history, confidence=cvar_confidence)
    if cvar_99 <= 0:
```

```
        cvar_99 = abs(pnl_history.min()) + 1e-6   # fallback: pior perda observada

    # f_base = E[ganho] / CVaR_99%
    f_cvar_base = mu_post / cvar_99

    # Quarter-Kelly sobre base CVaR-limitada
    f_quarter   = quarter_kelly_factor * f_cvar_base

    # Scaling continuo por P_meta (v10.5 -- inalterado)
    phi_z    = pmeta_scaling_factor(p_meta)
    f_scaled = f_quarter * phi_z

    # Cap final
    f_final = float(np.clip(f_scaled, 0.0, max_position_pct))

    return {
        'position_fraction' : f_final,
        'e_pnl'             : round(mu_post, 6),
        'cvar_99'           : round(cvar_99, 6),
        'f_cvar_base'       : round(f_cvar_base, 6),
        'f_quarter'         : round(f_quarter, 6),
        'phi_z'             : round(phi_z, 4),
        'p_meta'            : p_meta,
        'reason'            : 'ok',
    }

def position_size_final(capital_total: float, hrp_weight: float,
                        kelly_result: dict, usdt_available: float,
                        max_position_pct: float = 0.22) -> float:
    """Tamanho final em USDT = capital x hrp_weight x kelly_cvar_scaled."""
    f = kelly_result['position_fraction']
    raw = capital_total * hrp_weight * f
    return max(min(raw, capital_total * max_position_pct, usdt_available * 0.95),
0.0)
```

10.3 Comparativo: Kelly Plug-in vs. Kelly CVaR-Limitado

Cenário P&L histórico	E[P&L]	CVaR_99%	Kelly Plug-in (v10.6)	Kelly CVaR (v10.7)	Diferença
Mercado normal: distribuição t-student	2.0%	8%	~6.0%	2.0%/8%×0.25=6.25%	Similar — distribuiçã capturada
Crypto leptocúrtico: crash de 20% não observado	2.0%	15% (tail real)	~6.0% (ignora tail)	2.0%/15%×0.25=3.33%	−44% de tamanho – tail risk respeitado
Regime de choque: LUNA/FTX-like (−40% gap)	2.0%	35% (tail extremo)	~6.0% (ignora choque)	2.0%/35%×0.25=1.43%	−76% de tamanho – sobrevivên ao choque garantida

Por que o CVaR muda o comportamento em crypto

Kelly plug-in usa a distribuição t-student fittada na amostra histórica. Se o histórico não inclui um crash tipo FTX (que liquidou exchanges em 24h), a distribuição t-student nunca verá esse tail. CVaR_99% calculado sobre o P&L real da estratégia captura o que o backtester de fato viu nos piores dias — incluindo os dias em que Luna colapsou, FTX quebrou, e USDC desviou do peg. Esses eventos estão no P&L histórico mesmo que a distribuição paramétrica não os represente bem.

10.4 HRP + Tabela de Sizing por Correlação

```
def hrp_weights(returns_regime, method="complete"):
    from scipy.cluster.hierarchy import linkage,dendrogram; from
scipy.spatial.distance import squareform
    from sklearn.covariance import LedoitWolf
    cov=LedoitWolf().fit(returns_regime).covariance_; std=np.sqrt(np.diag(cov))
    corr=cov/np.outer(std,std); np.fill_diagonal(corr,1.0); dist=np.sqrt(0.5*(1-
corr))
    link=linkage(squareform(dist),method=method)
    order=dendrogram(link,no_plot=True)["leaves"]
    N=returns_regime.shape[1]; weights=np.ones(N)
    _recursive_bisection(cov,order,weights,0,N); return weights/weights.sum()
```

Condição	Posições Máx	Capital por Posição	Caixa Mínimo
avg_pairwise_corr < 0.60	4	HRP × Kelly_CVaR_scaled (máx 22%)	28%
avg_pairwise_corr 0.60–0.75	4	HRP × Kelly_CVaR_scaled (máx 15%)	40%
avg_pairwise_corr 0.75–0.85	3	HRP × Kelly_CVaR_scaled (máx 10%)	70%
avg_pairwise_corr > 0.85	0	—	100% caixa
Modo Defensivo (DD > 10%)	2	máx 10% — scaling CVaR+P_meta mantido	70%
Modo Crise (DD > 18%)	0	—	100% halt total
C2ST MODERATE	0 novas	Stop loss acelerado	Mínimo atual
C2ST SEVERE	0 halt total	Saída gradual 24h	100%

10.5 CVaR de Portfólio Conjunto antes da Execução (v10.9)

Correlação de Cauda = 1.0 em Pânico: CVaR Marginal Subestima Risco Real

Em pânico crypto, correlações cruzadas convergem a 1.0. 4 sinais com CVaR individual 5% cada produzem CVaR conjunto próximo a 20%, não soma ponderada dos individuais. HRP controla correlações normais mas não correlações de cauda (tail dependence). Protocolo v10.9:

simulador de P&L conjunto pós-HRP calculado antes de enviar ordens. Se CVaR_portfolio > limite global (15%), todos os multiplicadores são reduzidos proporcionalmente.

```
def compute_portfolio_cvar(active_signals: list,
                               hrp_weights: dict,
                               returns_history: pd.DataFrame,
                               kelly_fractions: dict,
                               capital_total: float,
                               portfolio_cvar_limit: float = 0.15,
                               cvar_confidence: float = 0.99) -> dict:
    """
    CVaR de portfólio conjunto para os ativos com sinais ativos.
    Executado ANTES de enviar ordens à Binance.

    active_signals: lista de ativos com sinal no dia
    hrp_weights: {ativo: peso_hrp}
    returns_history: DataFrame (N_dias x N_ativos) de retornos históricos diários
    kelly_fractions: {ativo: f_kelly_cvar_escalado} -- frações individuais já
calculadas
    capital_total: capital total em USDT
    portfolio_cvar_limit: CVaR máximo do portfólio conjunto (padrão: 15%)

    Retorna: {ativo: f_ajustada} -- frações ajustadas para respeitar limite
conjunto.
    """
    if not active_signals:
        return {}

    # Construir vetor de pesos do portfólio ativo
    ativo_weights = {}
    for ativo in active_signals:
        w_hrp = hrp_weights.get(ativo, 1.0 / len(active_signals))
        f_kelly = kelly_fractions.get(ativo, 0.0)
        # Fração efetiva do capital total alocada neste ativo
        ativo_weights[ativo] = w_hrp * f_kelly

    # Normalizar para somar 1.0 (apenas os ativos ativos)
    total_w = sum(ativo_weights.values())
    if total_w <= 0:
        return {a: 0.0 for a in active_signals}

    norm_weights = {a: w / total_w for a, w in ativo_weights.items()}

    # Calcular retornos históricos do portfólio ponderado
    available = [a for a in active_signals if a in returns_history.columns]
    if not available:
        return kelly_fractions  # fallback: sem dados históricos

    weights_arr = np.array([norm_weights.get(a, 0.0) for a in available])
    ret_matrix  = returns_history[available].dropna().values   # (T x N)

    # Retorno diário do portfólio: média ponderada dos retornos
    portfolio_returns = ret_matrix @ weights_arr

    # CVaR-99% do portfólio conjunto
```

```
    cvar_portfolio = compute_cvar(portfolio_returns, confidence=cvar_confidence)

    # Fração do capital total exposta
    exposure_fraction = total_w   # soma das frações efetivas

    # CVaR do portfólio em termos do capital total
    cvar_total_exposure = cvar_portfolio * exposure_fraction

    print(f"CVaR portfólio conjunto ({len(available)} ativos):
{cvar_portfolio:.2%}")
    print(f"Exposição total: {exposure_fraction:.2%} | CVaR efetivo:
{cvar_total_exposure:.2%}")

    # STRESS TEST ρ=1.0 (v10.10): cisne negro que nao esta no historico
    # CVaR_stress = Σ(HRP_weights × CVaR_individual) com correlacao perfeita
forçada
    cvar_stress_components = []
    for ativo in available:
        w      = norm_weights.get(ativo, 0.0) * exposure_fraction
        pnl_a  = returns_history[ativo].dropna().values
        cvar_a = compute_cvar(pnl_a, confidence=cvar_confidence)
        cvar_stress_components.append(w * cvar_a)
    cvar_stress = sum(cvar_stress_components)   # soma linear = ρ=1.0 perfeito
    print(f"CVaR stress (ρ=1.0): {cvar_stress:.2%} | Limite portfolio:
{portfolio_cvar_limit:.2%}")

    # Usar o MAIOR entre CVaR historico e stress scenario como criterio
    cvar_efetivo = max(cvar_total_exposure, cvar_stress)

    if cvar_efetivo <= portfolio_cvar_limit:
        # Portfólio dentro do limite em AMBOS os cenarios -- sem ajuste
        return kelly_fractions

    # CVaR excede limite: reduzir todos os multiplicadores proporcionalmente
    reduction_factor = portfolio_cvar_limit / cvar_efetivo
    print(f"CVaR efetivo {cvar_efetivo:.2%} > limite {portfolio_cvar_limit:.2%} "
          f"Reduzindo por fator {reduction_factor:.3f}")

    adjusted_fractions = {
        ativo: kelly_fractions.get(ativo, 0.0) * reduction_factor
        for ativo in active_signals
    }
    return adjusted_fractions

def position_size_final_v109(capital_total: float,
                             active_signals: list,
                             hrp_weights: dict,
                             kelly_fracs_individual: dict,
                             returns_history: pd.DataFrame,
                             usdt_available: float,
                             portfolio_cvar_limit: float = 0.15,
                             max_position_pct: float = 0.22) -> dict:
    """
    Pipeline completo v10.9:
    1. Kelly CVaR individual por ativo (v10.7)
    2. Scaling por P_meta calibrado pooled (v10.9)
    3. CVaR de portfólio conjunto -- reduzir se exceder limite (v10.9)
```

```
    4. Cap individual max_position_pct
    5. Retornar {ativo: usdt_amount} para execução na Binance
    """
    # Passo 3: ajuste por CVaR portfólio conjunto
    fracs_adjusted = compute_portfolio_cvar(
        active_signals, hrp_weights, returns_history,
        kelly_fracs_individual, capital_total, portfolio_cvar_limit
    )

    # Passo 4: calcular USDT por ativo com cap individual
    usdt_orders = {}
    for ativo in active_signals:
        f = float(np.clip(fracs_adjusted.get(ativo, 0.0), 0.0, max_position_pct))
        raw_usdt = capital_total * hrp_weights.get(ativo, 0.0) * f
        usdt_orders[ativo] = max(min(raw_usdt, capital_total * max_position_pct,
                                     usdt_available * 0.95 /
max(len(active_signals), 1)), 0.0)

    print(f"Ordens finais: {[(a, f'{v:.2f} USDT') for a,v in
usdt_orders.items()]}")
    return usdt_orders
```

Cenário	CVaR individual	CVaR portfólio histórico	CVaR stress (ρ=1.0)	Ação v10.10
1 sinal ativo	5%	5% (trivial)	5% (igual)	Sem ajuste
4 sinais, correlação normal (0.3)	5% cada	~8% (diversificação)	20% (soma linear)	Stress ativo: REDUÇÃO 15%/20%=0.75
4 sinais, correlação cauda (0.9)	5% cada	~19% (quase aditivo)	20% (soma linear)	REDUÇÃO pelo maior: 15%/20%=0.75
6 sinais, regime de crise	5% cada	~28%	30% (soma linear)	REDUÇÃO: 15%/30%=0.50 — stress domina

PARTE 11 — SAÍDAS E GESTÃO DE RISCO

11 Regras de Saída, Breakers e Custódia

11.1 Hierarquia de Saída

Prioridade	Gatilho	Ação	Timing
1	Posição cai -15% desde entrada	Fechar 100% a mercado	Próxima abertura
2	BTC_7d_return < -15%	Saída de todas via limit	Ao longo do dia
3	UPS entra no top 25%	Fechar posição afetada	Fechamento do dia
4	HMM: regime_ok False	Fechar todas	Fechamento dia seguinte
5	C2ST SEVERE (AUC > 0.70)	Saída gradual de todas (24h)	Ao longo de 24h
6	Sinal primário inverte	Fechar posição afetada	Fechamento dia seguinte
7 (time stop)	45 dias desde entrada	Fechar posição	Fechamento dia 45

11.2 Custódia

Parcela	Localização	Instrumento
Capital ativo	Exchange tier-1 (Binance/Coinbase)	Ativo conforme engine
Caixa operação (até 30%)	Exchange tier-1	USDC ou USDT
Reserva principal (> 30%)	Hardware wallet	USDC on-chain ou BTC
Regra concentração	Máx 40% por exchange	—
Retirada semanal	Todo domingo → cold wallet	—

11.2 Circuit Breaker: Corwin-Schultz + Funding Rate (v10.10)

A Cegueira Estrutural do C2ST e o Problema do OFI em Dados OHLCV

Dois problemas: (1) C2ST com production_window=20 dias: LUNA colapsou em 72h, FTX em 48h — primeiros 19 dias sem proteção. (2) OFI de barras OHLCV é ~90% ruído branco: OFI genuíno exige dados tick-by-tick (L1) para classificação bid/ask. Calcular OFI de barras de 4h (Close vs. Open) é essencialmente ruído. Corwin-Schultz (2012) extrai o spread implícito bid-ask de High/Low de barras adjacentes sem dados tick — detecta retirada de liquidez por market makers com dados OHLCV padrão.

```
import pandas as pd, numpy as np
from datetime import datetime

class CircuitBreakerCorwinSchultz:
    """
    Circuit Breaker v10.10: Corwin-Schultz Spread + Funding Rate.

    SUBSTITUI o OFI de barras OHLCV (v10.9) que era ~90% ruido branco.
    OFI genuino requer dados tick-by-tick (L1) para classificacao bid/ask.
    Calcular OFI de barras de 4h usando Close vs Open e simplesmente ruido.

    Corwin-Schultz (2012): extrai spread implícito bid-ask de High/Low de
    barras adjacentes. Nao requer dados tick. Funciona com OHLCV diario ou 4h.
    Um pico no spread implícito sinaliza que market makers retiraram liquidez
    do book -- exatamente o que acontece nas primeiras horas de um colapso.

    Combinado com Funding Rate extremo (independente, nao requer L1).
    """
    def __init__(self,
                 sigma_threshold : float = 3.0,
                 window_bars     : int   = 6,    # 6 barras de 4h = 24h de janela
                 history_bars    : int   = 90,   # 90 barras de referencia (15d em
4h)
                 halt_duration_h : int   = 24):
        self.sigma_threshold = sigma_threshold
        self.window_bars     = window_bars
        self.history_bars    = history_bars
        self.halt_duration   = halt_duration_h
        self.halt_active     = False
        self.halt_until      = None

    def corwin_schultz_spread(self, high: np.ndarray, low: np.ndarray) ->
np.ndarray:
        """
        Estimador de spread bid-ask de Corwin e Schultz (2012).

        Usa o ratio (H/L) de barras adjacentes para estimar o spread implícito.
        Intuitivo: se H/L e muito grande em relacao ao dia anterior, os market
        makers ampliaram o spread — sinal de retirada de liquidez.

        high, low: arrays de precos (N barras). Retorna spread estimado (N-1
valores).

        Referencia: Corwin e Schultz (2012), "A Simple Way to Estimate Bid-Ask
        Spreads from Daily High and Low Prices", Journal of Finance.
        """
        beta  = np.log(high[1:] / low[1:])**2 + np.log(high[:-1] / low[:-1])**2
        gamma = np.log(np.maximum(high[1:], high[:-1]) /
                       np.minimum(low[1:], low[:-1]))**2
        alpha = (np.sqrt(2 * beta) - np.sqrt(beta)) / (3 - 2*np.sqrt(2)) -
np.sqrt(gamma / (3 - 2*np.sqrt(2)))
        spread = 2 * (np.exp(alpha) - 1) / (1 + np.exp(alpha))
        return np.clip(spread, 0, 1)   # spread nao pode ser negativo

    def check_cs_spread_anomaly(self, spreads_recent: np.ndarray,
                                spreads_history: np.ndarray) -> tuple:
```

```
        """Spread Corwin-Schultz acima de sigma_threshold * sigma_hist?"""
        mu, std = spreads_history.mean(), spreads_history.std()
        if std < 1e-8:
            return False, 0.0
        z = (spreads_recent.mean() - mu) / std
        return z > self.sigma_threshold, round(z, 3)

    def check_funding_anomaly(self, funding_recent: np.ndarray,
                              funding_history: np.ndarray) -> tuple:
        """Funding Rate em anomalia > sigma_threshold?"""
        mu, std = funding_history.mean(), funding_history.std()
        if std < 1e-10:
            return False, 0.0
        z = abs(funding_recent.mean() - mu) / std
        return z > self.sigma_threshold, round(z, 3)

    def evaluate(self, high_bars: np.ndarray, low_bars: np.ndarray,
                 high_hist: np.ndarray, low_hist: np.ndarray,
                 funding_recent: np.ndarray, funding_history: np.ndarray,
                 timestamp=None) -> dict:
        """
        Avalia Circuit Breaker com Corwin-Schultz + Funding Rate.

        high_bars, low_bars: ultimas window_bars barras de 4h (para spread CS)
        high_hist, low_hist: history_bars barras de referencia expandida
        funding_recent: funding rate das ultimas window_bars barras
        funding_history: funding rate historico de referencia
        """
        from datetime import datetime, timedelta
        if timestamp is None:
            timestamp = datetime.utcnow()

        if self.halt_active and self.halt_until and timestamp < self.halt_until:
            return {'action':'HALT_ACTIVE','halt_active':True,
                    'reason':f'Circuit Breaker ativo até {self.halt_until}'}

        # Calcular spread Corwin-Schultz em janela recente e historico
        spread_recent  = self.corwin_schultz_spread(high_bars, low_bars)
        spread_history = self.corwin_schultz_spread(high_hist, low_hist)

        cs_trig, cs_z     = self.check_cs_spread_anomaly(spread_recent,
spread_history)
        fund_trig, fund_z = self.check_funding_anomaly(funding_recent,
funding_history)

        if cs_trig or fund_trig:
            self.halt_active = True
            self.halt_until  = timestamp + timedelta(hours=self.halt_duration)
            reasons = []
            if cs_trig:   reasons.append(f"Corwin-Schultz spread z={cs_z:.2f} >
{self.sigma_threshold}σ")
            if fund_trig: reasons.append(f"Funding z={fund_z:.2f} >
{self.sigma_threshold}σ")
            print(f"CIRCUIT BREAKER: {' | '.join(reasons)}")
            return {'action':'HALT_ALL','cs_z':cs_z,'funding_z':fund_z,
                    'halt_until':self.halt_until,'halt_active':True,
                    'reason':' | '.join(reasons)}
```

```
        return
{'action':'CONTINUE','cs_z':cs_z,'funding_z':fund_z,'halt_active':False}

# INTEGRAÇÃO NA PIPELINE DIÁRIA:
# ANTES de qualquer sinal ou ordem (usando barras de 4h da Binance):
# 1. Buscar últimas 12 barras de 4h (OHLCV) via Binance Spot/Futures API
# 2. Calcular Corwin-Schultz spread das ultimas 6 barras vs. histórico 90 barras
# 3. Buscar Funding Rate das últimas 8h via Binance Futures API
# 4. cb.evaluate(high_rec, low_rec, high_hist, low_hist, fund_rec, fund_hist)
# 5. Se HALT_ALL: parar tudo. Notificar. Aguardar halt_until.
# 6. Se CONTINUE: prosseguir para C2ST diário e execução.

# VANTAGEM vs. OFI:
# OFI: requer dados tick L1 (bid/ask classificados por agressor). Nao disponivel
em OHLCV.
# Corwin-Schultz: funciona com High/Low de barras OHLCV. Disponivel via Binance
REST API.
# Ambos detectam retirada de market makers -- CS via spread implícito, OFI via
volume direcional.

# CALIBRACAO:
# sigma_threshold=3.0: ~0.13% de falsos positivos em distribuicao normal
# window_bars=6: 6 barras de 4h = 24h de janela de monitoramento
# history_bars=90: 90 barras de 4h = 15 dias de referencia expandida
# Validar com LUNA mai/2022 e FTX nov/2022: spread CS deve ter disparado.
```

Evento histórico	CS Spread anomalia (horas antes)	Funding anomalia	Circuit Breaker teria acionado?
LUNA colapso (Mai/2022)	CS spread > +5σ em 8h (liquidez colapsou)	Funding +0.5% em 4h (short squeeze)	SIM — ~8h antes do depegging total
FTX bank run (Nov/2022)	CS spread > +4σ em 6h (market makers saíram)	Funding extremamente negativo em 8h	SIM — ~6h antes do halt das saques
Março 2020 COVID (BTC −40%)	CS spread > +6σ em 4h (book esvaziou)	Funding negativo extremo	SIM — ~4h antes do flash crash
Correção normal (−10-15%)	CS spread dentro de ±2σ	Funding dentro do normal	NÃO — sem halt (falso positivo evitado)

PARTE 12 — PROTOCOLO DE BACKTEST

12 Backtest Anti-Fraude + Anti-Drift: Checklist v10.4

12.1 Deflated Sharpe Ratio

```
def deflated_sharpe_ratio(sr_backtest,sr_benchmark,n_trials,n_obs,skew,kurt):
    from scipy.stats import norm
    E=((1-np.euler_gamma)*norm.ppf(1-1/n_trials)+np.euler_gamma*norm.ppf(1-
1/(n_trials*np.e)))
    sa=sr_backtest*(1-skew*sr_backtest+(kurt-1)/4*sr_backtest**2)**0.5
    prob=norm.cdf((sa-sr_benchmark)*np.sqrt(n_obs-1)/np.sqrt(1-skew*sa+(kurt-
1)/4*sa**2))
    return prob-norm.cdf(E)  # DSR>0: edge real
```

12.2 Checklist Completo v10.4

#	Verificação	Critério
1	Universo point-in-time (Luna, FTT, CEL obrigatórios)	Obrigatório
2	Sinais com dados disponíveis no dia histórico	Obrigatório
3	Entrada no fechamento do DIA SEGUINTE ao sinal	Obrigatório
4	Custo de 1.2% + cenário adverso 1.8%	Obrigatório
5	FracDiff: d* com expanding window estrita (nunca global)	v10.5 — Obrigatório
5b	FracDiff: np.log(prices) ANTES de qualquer FracDiff — não preços brutos	NOVO v10.7 — Obrigatório
5c	FracDiff: weight cutoff τ=1e-5 (não window=10) — janela efetiva ~500 obs	v10.6 — Obrigatório
5d	FracDiff: flag log_space=True salvo por ativo para auditoria	NOVO v10.7 — Auditoria
6	Triple-Barrier HLC: Low[t] ≤ SL verificado ANTES de High[t] ≥ TP	v10.8 — Obrigatório
6b	Slippage √(Q/V): exit_price = barrier_sl × (1 − η×σ×√(order/volume)) — não linear	NOVO v10.10 — Obrigatório
6c	pnl_real (com slippage) alimenta CVaR — não pnl_teórico	NOVO v10.9 — Obrigatório
6b	Triple-Barrier: distribuição +1/−1/0 equilibrada (~30-40% cada) com dados HLC	v10.5 — Diagnóstico
6b	N_eff dinâmico (t_touch real) calculado ANTES de treinar. Ganho vs. estático documentado	v10.6 — Obrigatório
6c	Pesos assimétricos: sl_penalty definido ANTES de ver dados de validação	v10.6 — Obrigatório
7	HMM: PCA com RobustScaler + Winsorização 1%/99% — não StandardScaler	NOVO v10.8 — Obrigatório

#	Verificação	Critério
7b	HMM: PC1 em Mar2020 e Nov2022 dentro de 3σ após Winsorização (não contaminado por crash)	NOVO v10.8 — Validação
7c	HMM: var_exp ≥ 80% em todas as janelas de treino expandidas	v10.7 — Obrigatório
7b	HMM: PC1 correlaciona com volatilidade macro? PC2 com funding/basis? (interpretabilidade)	NOVO v10.7 — Diagnóstico
7c	HMM+PCA: 2022 bear detectado em > 60% dos dias jan-jun/2022?	NOVO v10.7 — Validação
8	CFI com Variation of Information (não correlação linear)	v10.4 — Obrigatório
8b	P_bma gerado via Purged K-Fold (não walk-forward simples) — stacking ortogonal	NOVO v10.7 — Obrigatório
8c	Verificar: P_bma_pkf[t] nunca usou t em treino (cada ponto genuinamente OOS)	NOVO v10.7 — Auditoria
9	CPCV N=6, k=2 (15 trajetórias) com P_bma_pkf como feature — PBO < 10%	v10.7 — Obrigatório
10	DSR > 0 com n_trials HONESTO (≥ 5.000 incluindo cemitério de backtests v1.0→v10.8)	NOVO v10.8 — Obrigatório
10b	Documentar: features testadas + modelos descartados + versões anteriores → somar ao n_trials	NOVO v10.8 — Auditoria
11	ECE < 0.05 após calibração do meta-modelo	Obrigatório
12	Kelly CVaR-limitado: CVaR_99% calculado com P&L histórico real (não distribuição paramétrica)	v10.7 — Obrigatório
12b	P_meta calibrado isotonicamente POOLED por cluster VI — ECE < 0.05 pós-calibração	NOVO v10.9 — Obrigatório
12c	CVaR de portfólio conjunto calculado pós-HRP antes de enviar ordens	NOVO v10.9 — Obrigatório
12d	CVaR_portfolio > 15%: todos os multiplicadores reduzidos proporcionalmente	NOVO v10.9 — Critério
12c	Calibração isotônica fit em OOS (CPCV) — nunca em dados de treino	NOVO v10.8 — Obrigatório
12b	Sizing: P_meta=0.85 aloca ≥ 20% mais que P_meta=0.65. CVaR limita corretamente.	v10.7 — Verificação
13	C2ST: Block Bootstrap (blocos 5d). PR-AUC + Brier Score calculados	v10.5/v10.6 — Obrigatório
13b	Circuit Breaker: Corwin-Schultz spread (High/Low barras 4h) + Funding Rate	NOVO v10.10 — Obrigatório
13b2	CB Corwin-Schultz: barras de 4h da Binance (REST API). Não requer dados tick L1.	v10.10 — Dados
13c	Backtest CB: CS spread > 3σ em LUNA mai/2022, FTX nov/2022, Mar/2020	v10.10 — Validação
14	Validação alpha IAs: ortogonalização não-linear (LightGBM max_depth=2)	v10.4 — Obrigatório se IAs

#	Verificação	Critério
15	Surface de parâmetros: 500 combinações, ≥ 60% com Sharpe > 0.8	Obrigatório
16	Subperíodos 2020–2024 testados SEPARADAMENTE — ≥ 4 de 6 positivos	Obrigatório
17	Total de trades > 120 N_eff (dinâmico) no período completo	Mínimo estatístico

12.2 DSR Honesto: n_trials do Cemitério de Backtests (v10.8)

O n_trials=500 é Sub-representação do Viés de Seleção Real

Surface de parâmetros: 500 combinações finais. Mas todo o processo SNIPER v1.0→v10.8 envolveu milhares de decisões: features testadas e descartadas, modelos explorados, lógicas abandonadas, parâmetros ajustados entre versões. O Sharpe da v10.8 sobreviveu a um processo de seleção muito mais amplo do que 500 trials. Usar n_trials=500 no DSR é autoengano matemático. Protocolo v10.8: n_trials ≥ 5.000 como lower bound conservador do cemitério completo de hipóteses.

```
from scipy.stats import norm
import numpy as np

def compute_dsr_honest(sharpe_is: float, n_trials_final: int = 500,
                       n_trials_total: int = 5000, T: int = 1500,
                       skewness: float = -0.3, kurtosis: float = 5.0) -> dict:
    """
    DSR com n_trials HONESTO (cemiterio de backtests incluido).
    Exige DSR > 0.95 para n_trials_total -- nao apenas n_trials_final.

    n_trials_total: estimativa conservadora de TODAS as hipoteses testadas
    desde v1.0 ate versao atual. Inclui: features descartadas, modelos
    testados, logicas reescritas, parametros explorados entre versoes.
    Lower bound recomendado: 5000.

    Referencia: Lopez de Prado, AFML 2018, Cap. 8.
    """
    euler_g = 0.5772

    def sr_star(n_trials):
        e_max = (1-euler_g)*norm.ppf(1-1/n_trials) + euler_g*norm.ppf(1-
1/(n_trials*np.e))
        var_sr = (1 - skewness*1 + ((kurtosis-1)/4)*1) / (T-1)
        return np.sqrt(var_sr) * e_max

    def dsr(sr_is, n_trials):
        sr_s  = sr_star(n_trials)
        var_sr = (1 - skewness*sr_is + ((kurtosis-1)/4)*sr_is**2) / (T-1)
        return norm.cdf((sr_is - sr_s) / np.sqrt(var_sr))

    dsr_final = dsr(sharpe_is, n_trials_final)
    dsr_total = dsr(sharpe_is, n_trials_total)
```

```
    passed    = dsr_total > 0.95

    print(f"DSR surface (n={n_trials_final:5d}): {dsr_final:.4f} -- {'OK' if
dsr_final>0.95 else 'FAIL'}")
    print(f"DSR honesto (n={n_trials_total:5d}): {dsr_total:.4f} -- {'OK' if
dsr_total>0.95 else 'FAIL'}")
    if not passed:
        sr_needed = 0.0
        for s in np.linspace(0.5, 5.0, 1000):
            if dsr(s, n_trials_total) > 0.95:
                sr_needed = s; break
        print(f"Sharpe IS necessario para DSR_honesto > 0.95: {sr_needed:.2f}")
    return {'dsr_final': dsr_final, 'dsr_honesto': dsr_total, 'passed': passed}

# COMO ESTIMAR n_trials_total HONESTAMENTE:
# Features testadas e descartadas:           ~30 trials
# Modelos por versao (10 versoes x 5 arq.): ~50 trials
# Parametros por versao (10 x 200 configs): ~2000 trials
# Logicas descartadas (FracDiff vs RSI etc):  ~200 trials
# Surface final:                              ~500 trials
# Total conservador: 2780. Usar 5000 como lower bound.
# Se DSR_honesto < 0.95: o Sharpe IS nao e estatisticamente defensavel.
```

n_trials honesto	SR IS mínimo para DSR > 0.95 (T=1500)	Interpretação
500 (surface apenas)	≈ 1.10	Threshold baixo — viés de seleção sub-representado 10×
2.000	≈ 1.35	Honesto se SNIPER tem histórico de 2-3 versões
5.000 (protocolo v10.8)	≈ 1.55	Inclui cemitério completo v1.0→v10.8
10.000	≈ 1.70	Ultra-conservador. Qualquer anomalia real sobrevive.

PARTE 13 — MÓDULO DE EXECUÇÃO BINANCE

13 Módulo de Execução: Integração com a Binance

SEGURANÇA

- **API keys NUNCA no código:** .env obrigatório — nunca commitado no Git.
- **Permissões mínimas:** 'Spot Trading' apenas. NUNCA 'Withdrawal'.
- **Testnet obrigatório:** Capital real apenas na Fase 8.

```
class BinanceExecutor:
    def __init__(self, testnet=True):
        from binance.client import Client; from decouple import config

self.client=Client(config('BINANCE_API_KEY'),config('BINANCE_API_SECRET'),testnet=tes
tnet)

    def place_limit_buy(self, symbol, usdt_amount, price_offset_pct=0.001):
        price=float(self.client.get_symbol_ticker(symbol=symbol)['price'])
        qty=self.calculate_quantity(symbol, usdt_amount)
        return self._place_order(symbol=symbol,side='BUY',order_type='LIMIT',

quantity=qty,price=str(round(price*(1+price_offset_pct),8)),timeInForce='GTC')

    def halt_all(self, reason=""):
        # Chamado por DriftMonitoringPipelineV2 em C2ST SEVERE
        import logging; logging.getLogger("binance").critical(f"HALT_ALL: {reason}")
```

PARTE 14 — MÓDULO DE NOTIFICAÇÕES

14 Notificações: Email + Telegram com Filtros Granulares

Tipo de Evento	Gatilho	Contém (v10.4)	Prioridade
COMPRA	Decisão aprovada pelo Meta-Modelo	P_meta, preço, posição	Alta — imediato
VENDA	Execução de saída	Razão, preço, P&L	Alta — imediato
REGIME_CHANGE	HMM flip OU C2ST MILD/MODERATE	C2ST AUC, top_drifted_features, clusters KS	Alta — imediato
HALT	DD>18% OU C2ST SEVERE	C2ST AUC, ação requerida	Crítica — TODOS canais sem filtro
ERRO_EXECUCAO	Falha na Binance ou pipeline	Detalhe do erro	Alta — imediato
RELATORIO_DIARIO	20:00 BRT	Posições, P&L, C2ST AUC do dia, alpha_retido_pct IAs	Normal
RELATORIO_SEMANAL	Domingos 19:00 BRT	drift_log, retrain_log semanal	Normal

PARTE 15 — CRITÉRIOS DE INVALIDAÇÃO GLOBAL

15 Critérios de Invalidação: Definidos Antes de Operar

REGRA SOBERANA

Definidos ANTES do primeiro trade. NÃO alterados durante operação.

15.1 Por Componente

Componente	Critério de Rejeição	Consequência
fracdiff	NÃO melhora AUC vs rsi_14 em holdout	Manter rsi_14.
CFI com VI	Heatmap VI: todas features com VI > 0.45 (nenhum cluster claro)	Usar Permutation Importance simples — sem estrutura de cluster.
CFI com VI	< 3 clusters com imp_mean > 0.002	Sem edge. Redesenhar features.
HMM	F1 OOS treino < 0.45	Revisitar features de input do HMM.
GBM	AUC OOS < 0.55 walk-forward	Descartar GBM. Usar HMM + engines.
Meta-Modelo	PBO ≥ 10% ou AUC < 0.52 em > 30% das trajetórias	Usar P_bma > 0.65 direto.
DSR	DSR < 0 com n_trials real	Sharpe inflado. Redesenhar.
C2ST	AUC > 0.55 em > 15% dos dias históricos estáveis	Ajustar auc_threshold ou reference_window antes de capital real.
IAs (ortogonalização NL)	alpha_retido_pct < 0.15 OU r2_ortho > 0.80	Remover IAs. Custo de API não justificado.
Módulo Binance	Slippage real > 2x estimado por 60 dias	Revisar universo.

15.2 Sistema Completo

Critério	Threshold	Consequência
Sharpe backtest OOS	< 0.70	Descartar sistema.
Max Drawdown backtest	> 45%	Descartar sistema.
DSR	< 0	Sharpe inflado. Redesenhar.

Critério	Threshold	Consequência
Subperíodos positivos	< 4 de 6	Dependente de regime. Descartar.
Sharpe realizado (real)	< 0.60 após 24 meses	Encerrar. Sem ajuste.
Max DD real	> 40% em qualquer ponto	Encerrar imediatamente.

PARTE 16 — ROADMAP DE IMPLEMENTAÇÃO

16 Roadmap: Sequência Irreversível

Fase 1 — Dados e Universo (Semanas 1–3)

ENTREGÁVEL: Dataset histórico 2019–2025 com Luna, FTT, CEL. SQLite completo.

1. **1.1** APIs: CoinGecko + Binance + Deribit + Bybit + CryptoPanic + TokenUnlocks.
2. **1.2** SQLite: features, signals, trades, execution_log, drift_log, retrain_log, ia_reflections.
3. **1.3** Git com .gitignore para .env. Versionamento de prompts das IAs.

Fase 2 — HMM + CFI com VI + GBM (Semanas 4–9)

ENTREGÁVEL: P_bma OOS validado. Heatmap VI documentado. HMM calibrado walk-forward puro.

4. **2.1** fracdiff: d ótimo por ativo. Validar: melhora AUC vs rsi_14?
5. **2.2** HMM walk-forward: threshold no treino, aplicado cegamente no teste.
6. **2.3** Calcular matriz VI. Gerar heatmap. Definir vi_threshold ANTES do CFI completo.
7. **2.4** CFI com VI: clusters identificados e documentados. imp_mean por cluster.
8. **2.5** LightGBM walk-forward. AUC OOS > 0.55. Surface 500 parâmetros.
9. **2.6** BMA com pesos por accuracy OOS rolling 30 dias.

Fase 3 — IAs + Validação de Alpha Não-Linear (Semanas 10–12)

ENTREGÁVEL: Pipeline de IAs com reflexão. alpha_retido_pct documentado. Decisão: manter ou remover.

10. **3.1** Prompts Bull (Claude) + Bear (GPT-4o) com versionamento Git.
11. **3.2** Executar validate_ia_alpha_nonlinear(). Documentar r2_ortho e alpha_retido_pct.
12. **3.3** > 0.30: IAs aprovadas. < 0.15: remover. 0.15–0.30: decisão manual documentada.
13. **3.4** Pipeline de reflexão com campos alpha_retido_pct e r2_ortho_at_time.

Fase 4 — Meta-Labeling CPCV (Semanas 13–17)

ENTREGÁVEL: Meta-Modelo com N_eff, CPCV 15 trajetórias, PBO < 10%, ECE < 0.05.

14. **4.1** Dataset walk-forward. N_eff calculado ANTES de treinar. Pooling cross-ativo se necessário.
15. **4.2** CPCV N=6, k=2. Modelo por N_eff por janela. PBO e DSR com n_trials honesto.
16. **4.3** ECE < 0.05. Reliability diagram documentado.

Fase 5 — C2ST + KS Drift Monitor (Semanas 18–21)

ENTREGÁVEL: C2STDriftDetector backtestado. Falso positivo histórico < 15%.

17. **5.1** Implementar C2STDriftDetector com RF max_depth=3.
18. **5.2** Definir auc_threshold, reference_window, production_window ANTES de dados de produção.
19. **5.3** Backtest C2ST histórico: AUC < 0.55 em dias estáveis (2021-H1, 2024)? < 15% falsos.
20. **5.4** Diagnóstico: C2ST alertaria quantos dias antes do KS em 2022?
21. **5.5** Implementar KSClusterDiagnostic com clusters VI da Fase 2.
22. **5.6** Integrar DriftMonitoringPipelineV2 ao SNIPEROrchestrator e SNIPERNotifier.

Fase 6 — Binance + Notificações (Semanas 22–24)

23. **6.1** BinanceExecutor em testnet=True. halt_all() chamado por C2ST SEVERE.
24. **6.2** REGIME_CHANGE inclui C2ST AUC e top_drifted_features.
25. **6.3** RELATORIO_DIARIO inclui C2ST AUC. RELATORIO_SEMANAL inclui drift_log.

Fase 7 — Paper Trading (Meses 6–7)

26. **7.1** Pipeline completo em testnet por 60 dias. C2ST em monitoramento ativo.
27. **7.2** Verificar: C2ST não gera MODERATE/SEVERE em período estável de paper trading.
28. **7.3** Alpha incremental vs. BMA sozinho > 2% Sharpe?

Fase 8 — Capital Real Gradual (Mês 8+)

29. **8.1** BINANCE_TESTNET=false. IP whitelist. R$ 30k (15%) — P_meta > 0.70.
30. **8.2** Escalar para R$ 80k após 60 dias se Sharpe real ≥ 0.6. R$ 150k após 90 dias adicionais.
31. **8.3** Nunca > 75% antes de 12 meses de track record auditado.
32. **8.4** CQL Offline RL em shadow mode no Mês 18.

PARTE 17 — RISCOS NÃO RESOLVIDOS

17 Honestidade: O que o Sistema NÃO Protege

Risco	Por que persiste	Mitigação parcial
C2ST com N pequeno na janela de produção	production_window=20 dias pode ter < 30 obs	Verificar N antes do C2ST. Se insuficiente: KS apenas.
VI sensível a n_bins com N pequeno	Clustering pode variar com n_bins±2 para N<200	n_bins=min(10,sqrt(N)). Validar estabilidade com n_bins±2.
IAs com alpha_retido na zona ambígua (0.15–0.30)	Custo de API marginal — decisão não binária	Reavaliar a cada 6 meses com dados mais recentes.
C2ST bloqueia em rally antes de confirmar estrutural	AUC>0.55 por 3 dias pode bloquear entradas num bull market	7 dias de observação. MODERATE bloqueia novas entradas — não fecha posições abertas.
N pequeno no Meta-Modelo	Mesmo com pooling, N_eff pode ser insuficiente	Logística automática quando N_eff < 60.
HMM overfit em apenas 2 ciclos de crypto	Walk-forward puro ainda pode overfit com N histórico limitado	Retreinar HMM rolling a cada 6 meses. C2ST monitora features do HMM também.
Exchange failure	Nenhum sistema protege se a custódia falha	Máx 40% por exchange. Cold wallet > 30%.
Drawdown psicológico	Sistema em -12% gera pressão para intervenção manual	Pre-commitment escrito. Critérios definidos antes. Accountability partner.

CONCLUSÃO

SNIPER v10.4 em Síntese

O QUE ESTE SISTEMA É

- Edge causal: contrapartes estruturalmente forçadas
- Universo anti-survivorship bias (Luna, FTT, CEL obrigatórios)
- Fractional Differentiation preservando memória da série
- HMM como filtro primário — calibrado por walk-forward puro
- CFI com Variation of Information — detecta redundância não-linear
- Meta-Labeling com N_eff e modelo adaptado ao tamanho
- CPCV 15 trajetórias + PBO < 10% + DSR > 0
- C2ST multivariado: alarme primário de drift (detecta covariância)
- KS por cluster VI: diagnóstico secundário de drift
- Validação de alpha das IAs por ortogonalização não-linear
- Execução direta na Binance com stop loss automático
- Notificações Email + Telegram com filtros granulares

O QUE ESTE SISTEMA NÃO É

- Imune ao problema de N pequeno (Logística pode ser o melhor modelo)
- Garantido de detectar todos os drifts imediatamente (N mínimo para C2ST)
- Estável na VI com N muito pequeno (sensível a n_bins)
- Proteção contra exchange failure
- Um sistema onde distinção pânico/estrutural é sempre óbvia
- Um produto de captação antes de 18 meses de track record auditado

AVALIAÇÃO ACUMULADA v10.4 → v10.7

v10.4→v10.9: VI/CFI, C2ST, Triple-Barrier HLC, FracDiff (log+expanding+τ), Uniqueness dinâmica, Block Bootstrap C2ST, W_SL=2×W_TS, PCA+HMM robusto, Stacking ortogonal, Kelly CVaR, calibração isotônica pooled, DSR honesto, slippage linear, CVaR portfólio, CB OFI. v10.10: (1) Market impact √(Q/V) — ΔP = η×σ_intraday×√(order/volume); fisicamente correto segundo Kyle/Almgren-Chriss; (2) Corwin-Schultz spread como proxy de liquidez no Circuit Breaker — OFI de barras OHLCV é ruído, CS extrai spread implícito de High/Low; (3) Stress test ρ=1.0 além do CVaR histórico — CVaR_stress = Σ(HRP_w × CVaR_individual) previne subestimação de cauda; (4) Time-decay na calibração isotônica — sample_weight exponencial; score 0.85 em 2021 pesa ~5% do que em 2025.